

OFFICIAL ACADEMIC REFERENCE SPECIFICATION

September 2026 · Version 1.1.0 (Production Baseline)



B2B Operations Platform & Workstation

Intelligent Vacation Rental Operations, Multi-Channel Synchronization & Grounded Guest Assistance

Academic monograph, formal system conception, and software engineering specification for vacation-property operations in Tunisia (Sidi Bou Said, La Marsa, Gammarth).

PROJECT CONTEXT

Graduation Thesis / Software Engineering Internship

ENGINEERING TEAM

Razi Jemni & Platform Core Stream

TECHNICAL STACK

FastAPI · PostgreSQL · React · Celery · Redis · Docker

VERIFICATION & QUALITY

136 Backend Tests · 132 Frontend Tests (100% Pass)

DOCUMENT STRUCTURE

Table of Contents

Modular structure and sequential assembly order of the technical specification (18 chapters).

01	Project Overview	Page 3
02	Context and Existing-System Study	Page 6
03	Stakeholders and Needs Analysis	Page 9
04	Scope, Assumptions, and Boundaries	Page 11
05	Functional Requirements	Page 14
06	Non-Functional Requirements	Page 18
07	Use-Case Analysis	Page 21
08	Module Decomposition	Page 25
09	System Architecture	Page 29
10	Data Conception	Page 34
11	Chatbot and Integration Conception	Page 41
12	UML and Workflow Models	Page 46
13	Project Management and Team Organization	Page 52
14	Testing and Validation Strategy	Page 56
15	Risks, Constraints, and Mitigation	Page 61
16	Requirements Traceability Matrix	Page 64
17	References, Glossary, and Decision Log	Page 67
18	UI/UX Prototype Architecture & Interaction Specification	Page 71

CHAPTER 01 · TECHNICAL SPECIFICATION

Project Overview

Status: Draft for team and supervisor review

Project Identity

Name: Vayca

Former working name: VacayOps

Domain: Property technology, hospitality operations, and applied artificial intelligence

Primary market: Tunisia

Delivery context: Three-person internship project

Executive Summary

Vayca is a B2B web platform intended for vacation-property management agencies and independent property owners. Its purpose is to reduce operational fragmentation by bringing booking visibility, property knowledge, guest communication, chatbot assistance, and maintenance follow-up into one workspace.

The project addresses three recurring operational problems:

1. Booking information is distributed across multiple channels, creating delayed updates and conflict risk.
2. Guest questions arrive through messaging channels in several languages and require rapid, property-specific answers.
3. Maintenance issues are frequently coordinated through informal messages without structured assignment or status tracking.

The proposed MVP provides a portfolio calendar, automated calendar imports, conflict alerts, a unified guest-message workspace, a multilingual chatbot with controlled escalation, property records, maintenance tickets, and a management dashboard.

Product Positioning

Vayca is not a public accommodation marketplace. Guests discover and reserve properties through existing channels such as Airbnb, Booking.com, or direct contact with the owner. Vayca operates behind the scenes as the owner's or agency's operational workspace.

The same architecture supports both agencies and independent owners:

- An agency is represented as a company with multiple users and properties.
- An independent owner is represented as a company with one or a small number of users and properties.
- Subscription pricing may vary later without requiring a second product architecture.

Project Objectives

Academic Objectives

- Apply requirements engineering and UML-based conception to a real operational problem.
- Design and implement a multi-module web application as a coordinated team.
- Practice database modeling, API design, asynchronous processing, AI integration, security, testing, and deployment.
- Produce traceable documentation connecting needs, requirements, modules, implementation, and tests.

Functional Objectives

- Centralize properties, bookings, conversations, and maintenance work.
- Import calendar data and identify overlapping reservations.
- Answer safe guest questions using verified property and availability data.
- Escalate uncertain or sensitive conversations to staff.
- Structure maintenance issues as tickets with assignment and status history.
- Provide managers with a concise view of work requiring attention.

Quality Objectives

- Keep the interface understandable to users with different levels of technical experience.
- Isolate each company's information in a multi-tenant database.
- Avoid duplicate records when integrations resend the same event.
- Keep external-integration failures visible and recoverable.
- Ensure AI-generated output is grounded, controlled, and auditable.

Expected Contribution

The project's value is the integration of several operational workflows into a localized and accessible workspace. Its contribution is not the invention of calendar, messaging, or ticketing technology individually. It is the coherent combination of these capabilities around the daily work of small vacation-property operators in Tunisia.

Current Maturity

At this documentation baseline, the MVP implementation is functional across its core operational modules:

- Multi-tenant PostgreSQL persistence with versioned Alembic migrations.
- Manager and staff authentication, session management, permissions, and invitations.
- Property CRUD, operational knowledge directives, archiving, and live property detail integration.
- Portfolio calendar with manual/direct booking creation, asynchronous iCalendar feed synchronization via Celery, and automated booking conflict detection/acknowledgment.
- Unified communication inbox with deterministic WhatsApp simulator transport, multilingual grounded chatbot replies, manual staff takeover, and shared unread tracking.
- Maintenance ticketing board with contractor contact management, chatbot suggestion confirmation/rejection, immutable status audit history, and guest notification templates.

- Management supervision dashboard deriving live attention items, arrivals, and departures.
- Automated test verification comprising 70 backend tests and 54 frontend Vitest tests.

External boundaries remain clearly defined: WhatsApp messaging operates in local simulator mode, cloud hosting/backup restore drills remain pre-pilot milestones, and market discovery interviews are prepared as a reviewed protocol pending field execution.

CHAPTER 02 · TECHNICAL SPECIFICATION

Context and Existing-System Study

Status: Draft; market claims require continued field validation

Sector Context

Tourism is economically significant in Tunisia and accommodation operations are distributed across several major coastal regions. Tunisia's National Institute of Statistics reported 2024 internal tourism expenditure of 16,122.6 million TND and a direct tourism contribution of approximately 5.1% of national GDP. Official accommodation statistics also show substantial capacity in Nabeul-Hammamet, Sousse-Kairouan, Yasmine Hammamet, Monastir-Skanes, and other target regions [REF-INS-CST-2024] [REF-INS-ACCOMMODATION].

These figures support the relevance of hospitality software, but they do not by themselves prove demand for Vayca. Customer interviews, workflow observation, pilot usage, and willingness-to-pay evidence remain necessary.

Existing Operational Situation

The working hypothesis is that small agencies and independent owners commonly combine several tools:

- Airbnb, Booking.com, or similar platform dashboards for reservations;
- exported iCalendar feeds for basic availability synchronization;
- WhatsApp for guest and contractor communication;
- spreadsheets or notes for property information and operational follow-up;
- informal calls and messages for maintenance assignment.

This arrangement is flexible and familiar, but information is fragmented. The same property may have booking dates in one system, access instructions in another file, guest history in a message thread, and maintenance status known only to one staff member.

Observed or Expected Problems

Fragmented booking visibility

Staff must consult multiple sources to understand availability. Delayed imports, manual bookings, cancellations, and blocked dates can produce inconsistent views.

Slow or inconsistent guest response

Property-specific questions may arrive outside working hours and in different languages. Staff must locate the correct information before replying, increasing response time and the chance of error.

Unstructured maintenance coordination

An issue reported in a guest conversation may be forwarded manually to a contractor. Without a ticket and status history, responsibility, urgency, and completion are difficult to verify.

Limited operational overview

Managers need a concise view of arrivals, departures, conflicts, escalated conversations, and urgent tickets. Existing channel dashboards generally focus on their own bookings rather than the operator's complete portfolio workflow.

Existing Solution Categories

Booking-channel dashboards

These provide authoritative information for reservations originating on their own platforms. Their limitation for Vayca's use case is fragmentation across channels and limited support for internal maintenance workflow.

Property-management systems

Commercial property-management systems may offer channel management, messaging, reporting, and automation. A detailed competitor matrix must be maintained using current vendor evidence before making strong claims about language support, pricing, or local-market absence.

General messaging tools

WhatsApp is accessible to guests and contractors but does not create structured property, booking, or ticket relationships by itself.

General task-management tools

Task boards can track maintenance, but they require manual duplication of guest, property, and booking context.

Proposed Improvement

Vayca does not replace public booking channels. It connects operational information around them:

Booking channels -> Vayca calendar and conflict monitoring
WhatsApp messages -> Vayca conversation and chatbot workflow
Reported issue -> Vayca maintenance ticket and assignment history
All modules -> Vayca dashboard and alerts

Preliminary SWOT Analysis

Strengths	Weaknesses
Focused operational workflows	Small and inexperienced implementation team
Multilingual chatbot concept	Dependence on third-party services
Local market and terminology awareness	No validated production customer yet
Unified context across modules	Broad MVP if boundaries are not controlled
Opportunities	Threats
Independent-owner subscription tier	Restricted access to official channel APIs
Pilot partnerships with local agencies	WhatsApp approval and policy constraints
Expansion to related Maghreb markets	Incorrect chatbot responses harming trust

Opportunities	Threats
Direct booking pages as a future add-on	Established PMS vendors improving localization

CHAPTER 03 · TECHNICAL SPECIFICATION

Stakeholders and Needs Analysis

Status: Accepted working baseline

Stakeholder Map

Stakeholder	Relationship to Vayca	Primary needs
Property manager	Authenticated primary user	Portfolio control, alerts, users, properties, assignments
Staff member	Authenticated operational user	Messages, bookings, property details, maintenance follow-up
Independent owner	Authenticated manager of a small company account	Same core workflows with simpler scale and pricing
Guest	External person without a Vayca account	Fast and accurate WhatsApp responses
Contractor	External contact without a Vayca account	Clear assignment details through existing communication channels
Chatbot	Controlled system capability	Verified context, allowed actions, escalation rules
Booking channel	External system	Calendar data exchange through supported mechanisms
WhatsApp provider	External system	Message delivery and webhook events
Project supervisor	Academic/industry governance	Demonstrable methodology, traceability, and justified decisions
Development team	Builder and maintainer	Stable contracts, module boundaries, reproducible environment

User Profiles

Property manager

The manager oversees one or more properties and is accountable for availability, guest service, maintenance, and staff coordination. The interface should prioritize exceptions and decisions rather than technical configuration.

Staff member

Staff handle day-to-day operations. They require fast access to guest context, property information, arrivals, departures, and maintenance status. They should not access company-user management or sensitive integration configuration unless explicitly authorized.

Independent owner

The independent owner follows the manager workflow but has a smaller portfolio and often works alone. The architecture should not create a separate user type or application. Simpler pricing and onboarding are commercial decisions rather than separate software architecture.

Guest

The guest never logs into Vayca. The guest communicates through WhatsApp and receives answers or updates without learning the internal system.

Contractor

The contractor is stored as a company contact but does not authenticate. Staff choose the contractor and communicate assignment details manually through WhatsApp or telephone during the MVP.

Core User Needs

Manager needs

- Know what requires attention today.
- Trust the portfolio availability view.
- Identify booking overlaps quickly.
- Control staff access.
- Review escalated conversations.
- Track unresolved maintenance work.

Staff needs

- Find the correct property information quickly.
- See whether the chatbot or a human is handling a conversation.
- Take over a conversation safely.
- Create a ticket from a reported issue.
- Record contractor assignment and ticket progress.

Guest needs

- Receive prompt answers in an understandable language.
- Receive accurate property and availability information.
- Reach a human for sensitive, uncertain, or exceptional requests.
- Receive understandable updates about reported issues.

Usability Principles

1. Use familiar operational terms rather than implementation terms.
2. Put exceptions and required actions before secondary statistics.
3. Keep the main navigation limited to Dashboard, Calendar, Messages, Maintenance, Properties, and Settings.
4. Open property details and conversation threads as drill-down views.
5. Combine color, icon, and text for every important status.
6. Never require users to understand terms such as RAG, OTA, webhook, or Kanban.
7. Make risky chatbot actions require human confirmation.

CHAPTER 04 · TECHNICAL SPECIFICATION

Scope, Assumptions, and Boundaries

Status: Accepted working baseline; subject to reviewed change

MVP Definition

The MVP is considered complete when the selected workflows operate end to end using persistent data, authentication, authorization, validation, and tested failure handling. A visually complete prototype does not satisfy this definition by itself.

In-Scope Capabilities

Foundation

- Company-based multi-tenancy
- Manager and staff accounts
- Email/password authentication
- Role-based backend authorization
- Database migrations
- Reproducible Docker development environment

Properties and booking calendar

- Create and update properties
- Store property knowledge and operational instructions
- Add supported calendar-feed connections
- Import and refresh iCalendar data
- Store bookings and blocked periods
- Create manual/direct bookings
- Detect relevant date overlaps
- Display portfolio calendar and conflict alerts

Guest communication

- Receive WhatsApp messages through a real test integration or simulator
- Store conversations and messages
- Detect and respond in supported languages
- Answer safe property-information questions
- Check availability through backend data
- Escalate sensitive or uncertain requests
- Allow staff to take over and reply manually

Maintenance

- Create tickets linked to properties and optional bookings
- Suggest ticket creation from a guest issue
- Require staff confirmation before ticket creation or assignment
- Store contractor contacts
- Record assignment and ticket status history
- Send or prepare guest status updates

Supervision

- Dashboard of attention items and daily operations
- In-app alerts
- Team and company settings
- Booking-channel configuration

Explicitly Out of Scope

- Public property marketplace
- Guest accounts
- Contractor accounts
- Online payment processing
- Public search and checkout
- Native mobile applications
- Autonomous demand, competitor, occupancy, or last-minute dynamic pricing, and OTA price publishing
- Owner portal
- Advanced revenue analytics
- Automated owner PDF reports
- Expedia and VRBO integration
- Italian and German chatbot support
- Automated review-request sequences
- 360-degree virtual property tours

Working Assumptions

ID	Assumption	Consequence if false
ASM-01	Airbnb and Booking.com calendar feeds are available for pilot properties	A simulator or manual import becomes necessary
ASM-02	WhatsApp test integration can be used during development	Demonstration relies on the webhook simulator
ASM-03	Property managers accept a web application	Additional mobile-first work may be required

ID	Assumption	Consequence if false
ASM-04	French, English, and Arabic cover the MVP pilot	Language scope must be revised
ASM-05	Staff can manually contact contractors	Automated contractor notification becomes a new requirement
ASM-06	One company account can represent an independent owner	Separate account modeling would be required

Constraints

- Three-person team with limited prior group-development experience
- Internship schedule and academic-documentation obligations
- Dependence on third-party service availability and approval
- No guaranteed access to official booking-platform partner APIs
- Need to protect personal, access, and conversation data
- Need to keep the application usable for non-technical operators

Change-Control Rule

Any proposal that adds a new actor, public page, external dependency, database entity, or MVP integration must answer:

1. Which validated need does it address?
2. Which existing requirement changes?
3. Which module owns it?
4. What implementation and testing work does it add?
5. What is removed or delayed to preserve the schedule?

CHAPTER 05 · TECHNICAL SPECIFICATION

Functional Requirements

Status: Proposed baseline for backlog reconciliation

Requirement Convention

Each requirement has a stable identifier. Priority uses the MoSCoW convention:

- **Must:** required for the internship MVP
- **Should:** important but may use a reduced implementation
- **Could:** useful if schedule permits
- **Won't:** excluded from the current MVP

Authentication and Company Management

ID	Requirement	Priority	Acceptance summary
FR-AUTH-01	A manager shall create or activate a company account.	Must	Company and manager records are persisted
FR-AUTH-02	A user shall authenticate with email and password.	Must	Valid credentials produce an authenticated session; invalid credentials do not
FR-AUTH-03	The system shall hash passwords and never store plaintext passwords.	Must	Database contains only secure password hashes
FR-AUTH-04	A manager shall create, invite, deactivate, and view staff accounts.	Must	Changes persist and affect access
FR-AUTH-05	The backend shall restrict manager-only operations from staff users.	Must	Direct unauthorized API calls return a forbidden response
FR-AUTH-06	An authenticated user shall log out and invalidate or remove the active session credentials.	Must	Protected requests no longer succeed

Property Management

ID	Requirement	Priority	Acceptance summary
FR-PROP-01	A manager shall create a property belonging to the current company.	Must	Property persists and appears in the property list
FR-PROP-02	Authorized users shall view property details.	Must	Only the current company's properties are visible
FR-PROP-03	A manager shall edit property details and operational knowledge.	Must	Updated values reload correctly
FR-PROP-04	Property knowledge shall include check-in, Wi-Fi, parking, house rules, amenities, and emergency information as applicable.	Must	Information can be stored and retrieved for chatbot grounding
FR-PROP-05	A manager shall archive a property without deleting historical bookings, messages, or tickets.	Should	Archived property is hidden from normal active views but history remains

Channels, Bookings, and Calendar

ID	Requirement	Priority	Acceptance summary
FR-CAL-01	A manager shall add and update a supported calendar feed for a property.	Must	Valid configuration persists; invalid URL is rejected
FR-CAL-02	The system shall periodically import iCalendar events.	Must	New or changed events appear within the configured polling interval
FR-CAL-03	The system shall update or cancel existing imported records without creating duplicates.	Must	Re-importing the same feed remains idempotent
FR-CAL-04	Authorized users shall create manual/direct bookings.	Must	Manual booking blocks the selected dates internally
FR-CAL-05	The system shall detect relevant overlapping active bookings for the same property.	Must	True overlaps create a conflict; adjacent stays do not
FR-CAL-06	Authorized users shall view a multi-property calendar.	Must	Bookings render on correct properties and dates
FR-CAL-07	The calendar shall distinguish booking source and status using text plus visual cues.	Must	Meaning is understandable without color alone
FR-CAL-08	Managers and staff shall view conflict details and acknowledgement state.	Must	Conflict source, dates, and related bookings are visible
FR-CAL-09	The system shall record calendar refresh status and failures.	Must	Users can distinguish current, delayed, and failed feeds
FR-CAL-10	A manager shall configure direct-booking base, weekend, and seasonal pricing rules, and authorized operators shall receive a server-calculated recommendation.	Should	Rules persist per company property; a direct booking records an explicit approved price or explained override.

Guest Conversations and Chatbot

ID	Requirement	Priority	Acceptance summary
FR-MSG-01	The system shall receive incoming WhatsApp webhook events or equivalent simulator events.	Must	Valid event is accepted and persisted
FR-MSG-02	The system shall find or create the correct conversation for the guest and property context.	Must	Message appears in one appropriate conversation
FR-MSG-03	The system shall prevent duplicate storage of repeated external message events.	Must	Same external message ID is stored once
FR-MSG-04	Authorized users shall view conversation lists and ordered message history.	Must	Messages show sender, time, language, and delivery state
FR-MSG-05	The chatbot shall identify or use the guest's supported language.	Must	Response uses French, English, or Arabic as appropriate

ID	Requirement	Priority	Acceptance summary
FR-MSG-06	The chatbot shall answer safe property questions using the relevant property record.	Must	Test questions receive grounded answers without invented facts
FR-MSG-07	Availability answers shall be calculated by a backend availability function.	Must	Model output cannot override database result
FR-MSG-08	The chatbot shall escalate uncertain or sensitive requests instead of automatically answering.	Must	Complaint, refund, cancellation, payment, emergency, and uncertain cases are flagged
FR-MSG-09	Staff shall take over a conversation and reply manually.	Must	Chatbot sending is disabled while staff takeover is active
FR-MSG-10	The system shall record whether each outgoing message was generated by the chatbot or written by staff.	Must	Source is auditable
FR-MSG-11	The chatbot may suggest a maintenance ticket from a reported issue.	Should	Structured suggestion is shown without creating a ticket automatically
FR-MSG-12	Staff shall confirm, edit, or reject a chatbot ticket suggestion.	Should	Only confirmation creates the ticket
FR-MSG-13	The chatbot may provide an owner-configured external booking link after checking availability.	Could	No booking is created by Vayca from the guest interaction

Maintenance

ID	Requirement	Priority	Acceptance summary
FR-TKT-01	Authorized users shall create a ticket linked to a property and optionally a booking.	Must	Required fields validate and persist
FR-TKT-02	A ticket shall have a defined priority and lifecycle status.	Must	Status uses Open, Assigned, In Progress, Resolved, or Cancelled as approved
FR-TKT-03	A manager or authorized staff member shall maintain contractor contacts.	Must	Contractors are contacts and cannot authenticate
FR-TKT-04	Staff shall assign a contractor manually and record the assignment.	Must	Assignment history contains contractor, actor, and time
FR-TKT-05	Authorized users shall update ticket status.	Must	Valid transition persists and invalid transition is rejected
FR-TKT-06	Authorized users shall view maintenance work by status and property.	Must	Board/list reflects database state
FR-TKT-07	The system shall preserve ticket and assignment history after resolution.	Must	Historical record remains accessible
FR-TKT-08	Staff may send a guest update when a linked ticket changes status.	Should	Message is reviewed or generated from an approved template

Dashboard and Alerts

ID	Requirement	Priority	Acceptance summary
FR-DASH-01	The dashboard shall show items requiring attention.	Must	Conflicts, escalated conversations, and urgent tickets are visible

ID	Requirement	Priority	Acceptance summary
FR-DASH-02	The dashboard shall show today's arrivals and departures.	Must	Values are calculated from active booking data
FR-DASH-03	The dashboard shall show basic portfolio occupancy states.	Should	Values are derived, not hard-coded
FR-DASH-04	Users shall open the relevant workflow from each dashboard item.	Must	Alert links to the correct booking, conversation, or ticket

Settings and Integration Visibility

ID	Requirement	Priority	Acceptance summary
FR-SET-01	Managers shall view and update company settings.	Must	Staff access is forbidden
FR-SET-02	Managers shall view booking-channel connection status.	Must	Status reflects real configuration and last refresh
FR-SET-03	Managers shall view WhatsApp integration mode and health.	Should	UI distinguishes simulator, test, and production modes
FR-SET-04	The interface shall not display an integration as connected when it is simulated or unconfigured.	Must	Demonstration state is truthful

CHAPTER 06 · TECHNICAL SPECIFICATION

Non-Functional Requirements

Status: Proposed measurable baseline

Security and Privacy

ID	Requirement	Verification target
NFR-SEC-01	All protected API operations shall require authenticated identity.	Automated authorization tests
NFR-SEC-02	Every tenant-owned query shall enforce company isolation.	Cross-company access tests return no data or forbidden response
NFR-SEC-03	Passwords shall use a modern password-hashing algorithm such as Argon2.	Hash format inspection and authentication tests
NFR-SEC-04	Secrets shall be loaded from environment or secret storage and excluded from Git.	Repository scan contains no real credentials
NFR-SEC-05	Production communication shall use HTTPS.	Deployment configuration review
NFR-SEC-06	Sensitive property data shall be returned only to authorized users and controlled chatbot workflows.	Permission and chatbot-grounding tests
NFR-SEC-07	Authentication and sensitive endpoints shall apply rate limiting or equivalent abuse protection before production use.	Repeated request test
NFR-SEC-08	Security verification shall use a documented subset of OWASP ASVS guidance.	Security checklist in test report

Tunisia's personal-data framework includes Organic Law No. 2004-63 and INPDP declaration or authorization procedures for relevant processing purposes. Final legal compliance must be reviewed with a qualified person before production deployment [REF-INPDP-LAW] [REF-INPDP-FORMS].

Reliability and Data Integrity

ID	Requirement	Verification target
NFR-REL-01	Reprocessing the same external event shall not duplicate persistent records.	Idempotency tests
NFR-REL-02	Integration failures shall be logged with a recoverable state.	Failure simulation
NFR-REL-03	Calendar refresh failure shall not delete the last known booking data.	Feed outage test
NFR-REL-04	Database updates involving related records shall use transactions.	Integration tests and code review
NFR-REL-05	Critical status changes shall record actor and timestamp.	Audit-field verification
NFR-REL-06	Backup and restore procedures shall be documented before pilot use.	Successful restore exercise

Performance

ID	Requirement	Initial target
NFR-PERF-01	Common authenticated pages should become usable promptly under pilot load.	95% of normal API requests under 1 second in local/pilot conditions, excluding external providers
NFR-PERF-02	Webhook endpoints shall acknowledge valid events quickly and defer slow processing.	Response under 2 seconds under test conditions
NFR-PERF-03	Calendar import shall process asynchronously without blocking user requests.	Worker execution test
NFR-PERF-04	The application shall support the agreed pilot portfolio without manual partitioning.	Load test using at least 100 properties and representative records

Performance targets are initial engineering targets, not service-level guarantees. They should be revised after measurement.

Usability and Accessibility

ID	Requirement	Verification target
NFR-UX-01	The main navigation shall contain six plain-language destinations.	Interface review
NFR-UX-02	Important statuses shall use text, icon, and color rather than color alone.	Accessibility review
NFR-UX-03	Core workflows shall remain usable on common mobile widths.	Tests at 360px, 390px, and 768px widths
NFR-UX-04	Forms shall provide explicit labels, validation messages, and recovery guidance.	Manual and automated UI tests
NFR-UX-05	Empty, loading, success, and failure states shall be designed for data-driven pages.	State checklist per page
NFR-UX-06	Technical terms shall be replaced or explained in user-facing interfaces.	Terminology review with target users

Maintainability

ID	Requirement	Verification target
NFR-MNT-01	The repository shall have one authoritative architecture baseline and one authoritative design system.	Documentation review
NFR-MNT-02	Database changes shall use version-controlled Alembic migrations.	Migration history review
NFR-MNT-03	Frontend and backend shall share documented API schemas through OpenAPI or generated types where practical.	Contract review
NFR-MNT-04	Each feature branch shall remain limited to one reviewable issue or tightly related unit.	Pull-request review
NFR-MNT-05	Automated tests shall cover critical domain rules.	CI test report

ID	Requirement	Verification target
NFR-MNT-06	Logs shall include correlation identifiers for external events where practical.	Log inspection

Portability and Deployment

ID	Requirement	Verification target
NFR-DEP-01	A documented Docker workflow shall start the development stack from a clean clone.	Clean-machine or clean-environment test
NFR-DEP-02	Environment-specific configuration shall not require source-code changes.	Development/test configuration comparison
NFR-DEP-03	Health checks shall distinguish API, database, Redis, and worker readiness.	Container health test
NFR-DEP-04	Deployment shall support database migrations as an explicit release step.	Staging deployment test

Chatbot Quality and Safety

ID	Requirement	Verification target
NFR-AI-01	The chatbot shall not invent property facts or availability.	Grounded-answer evaluation set
NFR-AI-02	Sensitive and uncertain messages shall prefer escalation over unsupported response.	Edge-case evaluation set
NFR-AI-03	Chatbot outputs and decisions shall be auditable.	Stored source, model/config version, and escalation reason
NFR-AI-04	French, English, Modern Standard Arabic, and representative Tunisian Arabic inputs shall be evaluated.	Language test dataset
NFR-AI-05	Model selection shall be based on measured quality, latency, and cost rather than brand assumption.	Recorded evaluation comparison
NFR-AI-06	A provider outage shall not prevent staff from handling conversations manually.	Provider failure test

CHAPTER 07 · TECHNICAL SPECIFICATION

Use-Case Analysis

Status: Proposed baseline

Use-Case Catalogue

ID	Use case	Primary actor	Supporting actor/system
UC-AUTH-01	Log in	Manager or staff	Authentication service
UC-TEAM-01	Manage staff	Manager	Email service, optional
UC-PROP-01	Manage property	Manager	Database
UC-CAL-01	Connect calendar feed	Manager	Booking platform feed
UC-CAL-02	Synchronize calendar	System	Worker, external feed
UC-CAL-03	Create manual booking	Manager or staff	Conflict service
UC-CAL-04	Review booking conflict	Manager or staff	Alert service
UC-MSG-01	Receive guest message	Guest	WhatsApp provider
UC-MSG-02	Answer safe guest question	Chatbot	Property and availability services
UC-MSG-03	Escalate conversation	Chatbot/system	Staff
UC-MSG-04	Take over conversation	Staff	WhatsApp provider
UC-TKT-01	Suggest maintenance ticket	Chatbot	Staff
UC-TKT-02	Create maintenance ticket	Manager or staff	Property/booking service
UC-TKT-03	Assign contractor	Manager or staff	Contractor directory
UC-TKT-04	Update ticket status	Manager or staff	Guest-message service, optional
UC-DASH-01	Review daily operations	Manager or staff	All operational modules

UC-AUTH-01: Log In

Preconditions: User account exists and is active.

Trigger: User submits email and password.

Main flow

1. The system validates the input format.
2. The system locates the user account.
3. The system verifies the password hash.
4. The system creates signed authentication credentials.
5. The system returns the user's identity, company, and role context.
6. The application opens the Dashboard.

Alternative flows

- Invalid credentials: show a generic authentication error.
- Inactive account: deny access and provide a contact instruction.
- Excessive failed attempts: apply rate limiting.

Postcondition: An authenticated session exists for an active user.

UC-CAL-02: Synchronize Calendar

Preconditions: An active property and valid calendar-feed configuration exist.

Trigger: Scheduled worker run or authorized manual refresh.

Main flow

1. The worker creates a synchronization-run record.
2. The worker downloads the calendar feed.
3. The parser validates the iCalendar content according to supported RFC 5545 elements [REF-IETF-ICAL].
4. Each supported event is normalized into booking or blocked-period data.
5. Existing records are matched by channel and external event identity.
6. New records are created and changed records are updated.
7. Missing or cancelled events follow the approved cancellation policy.
8. Conflict detection evaluates changed active periods.
9. The run records success, counts, and completion time.

Alternative flows

- Feed unavailable: retain previous data and mark refresh failed.
- Invalid event: record the event error and continue where safe.
- Duplicate delivery: update the matched record without duplication.

Postcondition: The database reflects the latest successfully interpreted feed state.

UC-CAL-04: Review Booking Conflict

Preconditions: Conflict detection has identified overlapping active bookings.

Trigger: User opens a conflict alert.

Main flow

1. The system displays the property, dates, booking sources, and current statuses.
2. The user verifies the source-channel information.
3. The user acknowledges or records the conflict resolution externally.
4. The system records the acknowledgement and actor.

Vayca does not automatically cancel or modify a source-platform booking during the MVP.

UC-MSG-02: Answer Safe Guest Question

Preconditions: The guest message is linked to a property or reliably resolved booking context.

Trigger: A supported incoming message is stored.

Main flow

1. The system detects language and intent.
2. The system classifies the request as safe, sensitive, or uncertain.
3. For property information, the backend retrieves authorized property knowledge.
4. For availability, the backend calculates availability from active booking data.
5. The chatbot produces a response constrained by retrieved facts.
6. Safety checks verify that the response is eligible for automatic sending.
7. The response is sent and stored with chatbot provenance.

Alternative flows

- Missing property context: escalate.
- Missing knowledge: state that staff will confirm and escalate.
- Sensitive intent: do not auto-send; escalate.
- Provider failure: preserve the incoming message for staff handling.

UC-MSG-04: Take Over Conversation

1. Staff opens an escalated or active conversation.
2. Staff activates manual-reply mode.
3. Automatic chatbot sending is disabled for that conversation.
4. Staff sends a reply.
5. The system stores and delivers the message.
6. Staff may return the conversation to automatic mode after review.

UC-TKT-01: Suggest Maintenance Ticket

1. The chatbot detects a probable property issue.
2. The chatbot extracts a proposed category, description, and priority.
3. The system displays the suggestion to staff.
4. Staff confirms, edits, or rejects the suggestion.
5. Only confirmation creates a ticket.

UC-TKT-03: Assign Contractor

1. Staff opens an unassigned ticket.
2. Staff selects an active contractor contact.
3. Staff confirms the assignment.
4. The system records assignment history and sets the ticket to Assigned.

5. Staff contacts the contractor manually through WhatsApp or telephone.

Contractors do not authenticate during the MVP.

CHAPTER 08 · TECHNICAL SPECIFICATION

Module Decomposition

Status: Accepted product boundaries; team allocation remains adjustable

Decomposition Principles

- A module owns a coherent business responsibility.
- Cross-module data access occurs through documented services or APIs.
- Shared foundation code remains small and intentional.
- Module ownership does not remove the need for team review.
- Frontend screens follow business modules rather than becoming a separate undocumented architecture.

Module 0: Platform Foundation

Responsibilities

- Application configuration
- Company tenancy
- Users, authentication, and authorization
- Database connection and migrations
- Shared API errors and pagination
- Audit conventions
- Docker development stack
- Deployment and observability baseline

Primary entities

Company , AppUser

Dependencies

PostgreSQL, FastAPI, SQLAlchemy, Alembic

Module 1: Properties and Calendar

Responsibilities

- Property records and operational knowledge
- Booking-channel configuration
- Calendar-feed synchronization
- Booking and blocked-period normalization
- Manual bookings
- Availability calculation

- Conflict detection
- Portfolio calendar and conflict views

Primary entities

Property , Channel , Booking , CalendarSyncRun , BookingConflict

Exposed services

- Property lookup
- Availability check
- Booking search
- Conflict query

Module 2: Guest Communication and Chatbot

Responsibilities

- WhatsApp webhook ingestion
- Conversation resolution
- Message persistence and delivery status
- Language and intent classification
- Property-grounded answers
- Availability-tool use
- Escalation
- Human takeover
- Maintenance-ticket suggestions

Primary entities

Conversation , Message , optional structured ActionSuggestion

Dependencies

Property lookup, availability service, WhatsApp provider, AI model provider

Module 3: Maintenance Operations

Responsibilities

- Ticket creation
- Priority and lifecycle
- Contractor contact directory
- Manual assignment
- Assignment history
- Guest update preparation or sending
- Maintenance board and filters

Primary entities

Ticket , Contractor , TicketAssignment , optional TicketStatusHistory

Dependencies

Property and optional booking context, conversation/message service

Module 4: Supervision and Settings

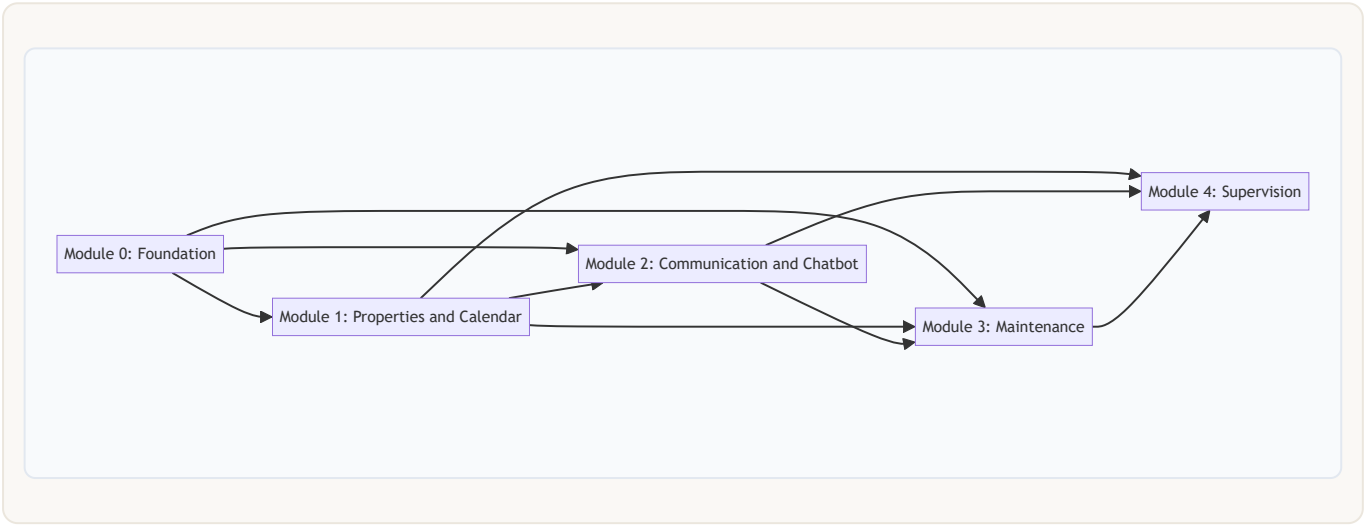
Responsibilities

- Attention-focused dashboard
- Daily arrivals and departures
- Conflict, escalation, and urgent-ticket alerts
- Company settings
- Team management interface
- Integration-health visibility

Primary data

Read models or queries derived from Modules 0-3

Dependency Direction



Suggested Team Allocation

This is a planning proposal, not a permanent ownership rule.

Member focus	Lead responsibilities	Support responsibilities
Platform lead	Module 0, deployment, shared backend standards	Review tenancy and security across modules
Calendar lead	Module 1 end to end	Shared frontend components and dashboard calendar data

Member focus	Lead responsibilities	Support responsibilities
Guest-operations lead	Module 2, then Module 3 with team support	AI evaluation and WhatsApp integration

Module 3 should be shared if Module 2 integration work becomes heavy. Module 4 should be assembled collaboratively after the source modules expose stable data.

Recommended Delivery Order

1. Foundation contracts and first migration
2. Authentication and company isolation
3. Property CRUD and knowledge
4. Manual bookings and availability
5. iCalendar import and conflicts
6. Conversations and message simulator
7. Chatbot grounding and escalation
8. Maintenance workflow
9. WhatsApp test integration
10. Dashboard and integration polish

CHAPTER 09 · TECHNICAL SPECIFICATION

System Architecture

Status: Proposed logical architecture; implementation details may evolve

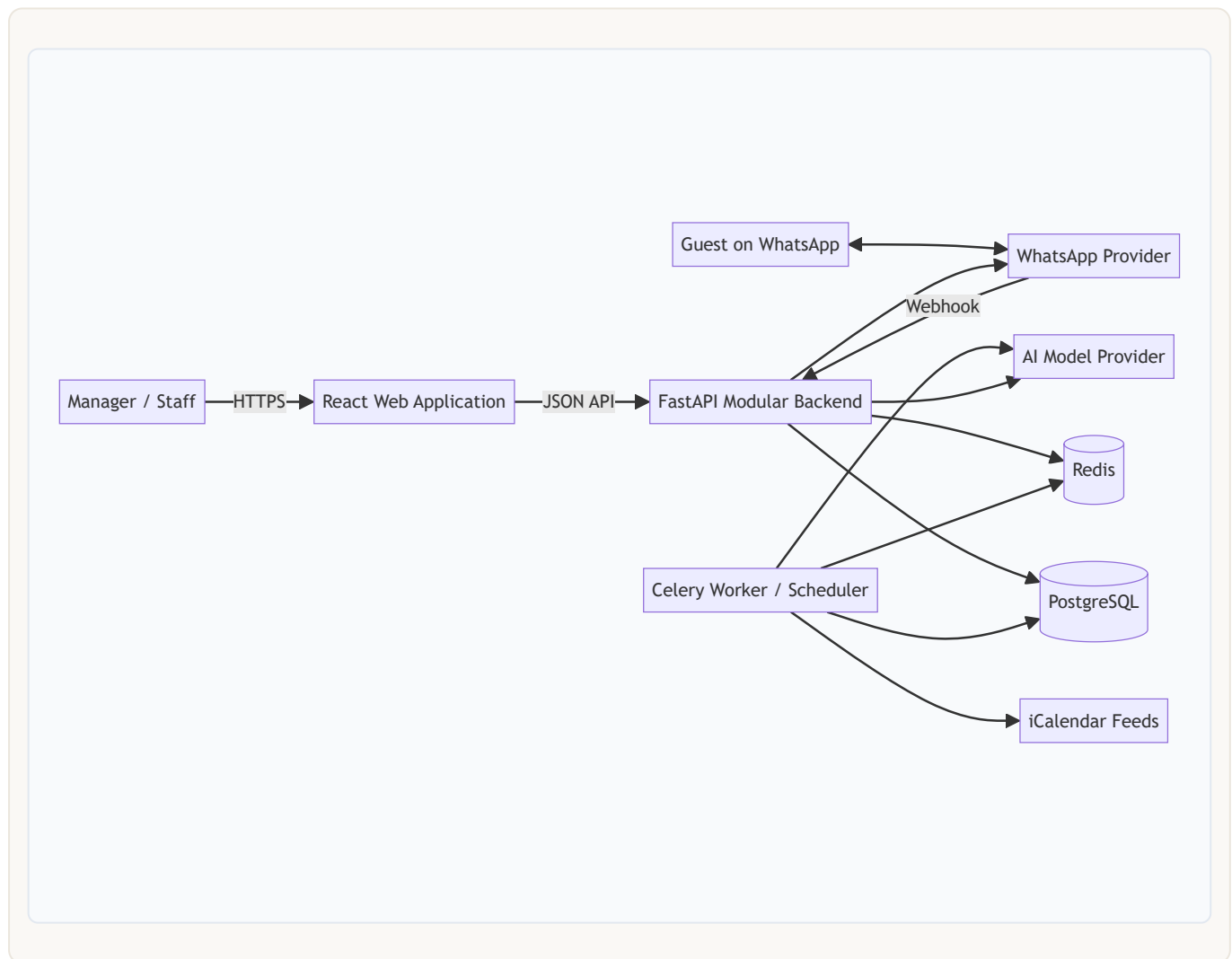
| Architectural Style

Vayca should begin as a modular monolith rather than a microservice system. A modular monolith is one deployable backend application with explicit internal module boundaries. This approach reduces deployment and debugging overhead for a three-person team while preserving a path to later service extraction if scale justifies it.

The system uses:

- React and TypeScript for the web client;
- FastAPI for HTTP APIs and webhook endpoints;
- PostgreSQL for persistent relational data;
- SQLAlchemy for persistence mapping;
- Alembic for version-controlled schema migrations [REF-ALEMBIC];
- Redis and Celery for scheduled and asynchronous work;
- an AI model provider for chatbot generation and classification;
- WhatsApp Cloud API or a provider-compatible adapter for messaging;
- iCalendar feeds for initial calendar interoperability.

Container-Level View



Backend Layers

API layer

- HTTP routing
- Request validation
- Authentication dependencies
- Authorization checks
- Response schemas
- Webhook acknowledgement

Application layer

- Use-case orchestration
- Transaction boundaries
- Cross-module coordination
- Background-job dispatch

- Chatbot action policy

Domain layer

- Availability rules
- Booking-overlap logic
- Ticket lifecycle rules
- Conversation state
- Roles and permissions

Infrastructure layer

- SQLAlchemy repositories
- External provider adapters
- Celery tasks
- Logging
- Configuration

Suggested Backend Structure

```
backend/app/  
|-- main.py  
|-- core/  
|   |-- config.py  
|   |-- database.py  
|   |-- security.py  
|   `-- errors.py  
|-- modules/  
|   |-- identity/  
|   |-- properties/  
|   |-- calendar/  
|   |-- messaging/  
|   |-- chatbot/  
|   |-- maintenance/  
|   `-- dashboard/  
|-- integrations/  
|   |-- ical/  
|   |-- whatsapp/  
|   `-- ai/  
|-- workers/  
|-- migrations/  
`-- tests/
```

This is guidance for discussion. The database lead may refine names while preserving module ownership and dependency direction.

Frontend Architecture

The frontend should mirror user workflows:

```
frontend/src/  
|-- app/  
|-- components/  
|-- features/  
|   |-- auth/  
|   |-- dashboard/  
|   |-- calendar/  
|   |-- messages/  
|   |-- maintenance/  
|   |-- properties/  
|   |-- settings/  
|-- services/  
|-- types/  
|-- styles/
```

Recommended user-facing navigation:

1. Dashboard
2. Calendar
3. Messages
4. Maintenance
5. Properties
6. Settings

Property details and conversation threads are drill-down routes, not main navigation destinations.

Authentication Architecture

The accepted MVP uses email/password authentication with:

- password hashing through `pwdlib` and Argon2id;
- random opaque session tokens with only token hashes stored in PostgreSQL;
- an `HttpOnly`, `SameSite=Lax` session cookie and a separate CSRF token;
- eight-hour configurable expiration and immediate revocation on logout or deactivation;
- Redis-backed failed-login limiting by normalized email and client IP;
- reusable FastAPI dependencies for active user, manager role, company context, and CSRF validation;
- manager and staff role checks;
- company context resolved for every authenticated request.

FastAPI's security guidance supports the selected Argon2 password-hashing library [REF-FASTAPI-SECURITY]. Vayca deliberately uses revocable server-backed sessions instead of browser-stored JWT credentials. Decision 0003 records this choice. Production requires HTTPS, secure cookies, and a real invitation-email adapter.

Multi-Tenant Request Rule

For every authenticated operation:

1. Hash the presented session token and load its unexpired, unrevoked record.

- 2. Load the active user.
- 3. Resolve `company_id` and role from trusted database state.
- 4. Apply role authorization.
- 5. Restrict every tenant-owned query to that company.
- 6. Record actor identity for sensitive changes.

Client-supplied `company_id` must never be trusted as authorization.

Background Processing

Background jobs are appropriate for:

- scheduled calendar-feed refresh;
- feed parsing and conflict checks;
- chatbot processing when webhook acknowledgement must remain fast;
- outgoing message retry;
- integration-health checks;
- future reporting.

Jobs must be idempotent because task systems and webhooks may retry delivery.

API Contract Principles

- Use versioned API routes such as `/api/v1`.
- Use consistent Pydantic request and response schemas.
- Use one documented error shape.
- Distinguish `401 Unauthorized` from `403 Forbidden` correctly.
- Provide pagination for growing lists.
- Return machine-readable status values and user-friendly UI labels separately.
- Generate OpenAPI documentation for frontend alignment.

Deployment Environments

Environment	Purpose	Data
Development	Local feature work	Local/mock/test data
Test/CI	Automated verification	Disposable database
Staging	Integrated demonstration	Controlled non-production data
Production/pilot	Real customer pilot	Protected real data

The internship may use development and staging first, but configuration should not assume only one environment.

CHAPTER 10 · TECHNICAL SPECIFICATION

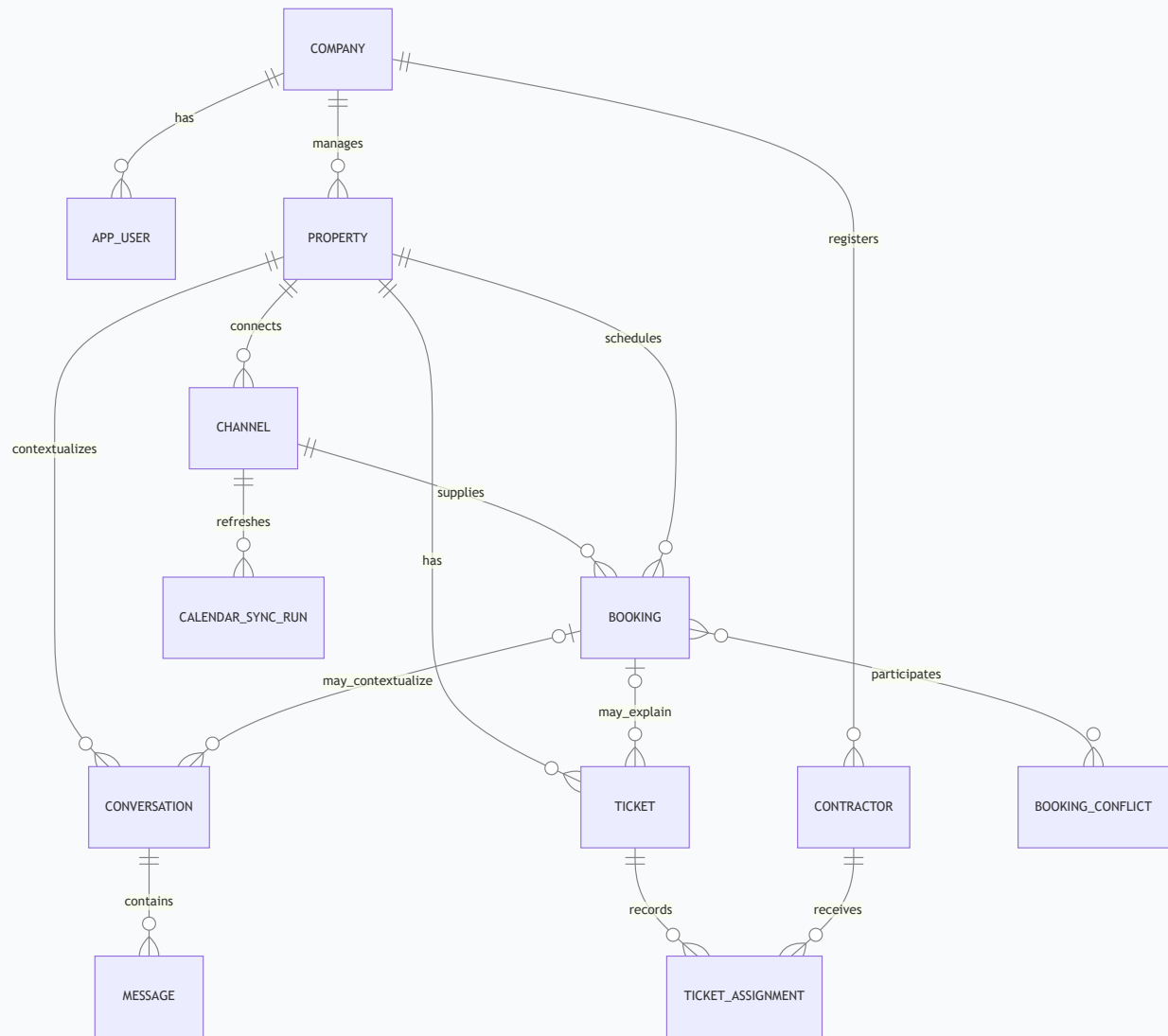
Data Conception

Status: Provisional domain model; initial schema baseline implemented, final DDL and workflow semantics remain subject to team review

| Modeling Goals

- Represent one or many properties under a company account.
- Preserve booking, conversation, and maintenance history.
- Support reliable external-event synchronization.
- Enforce tenant isolation and key integrity.
- Provide auditable chatbot and staff actions.
- Avoid unnecessary marketplace, payment, guest-account, or contractor-login entities.

Conceptual Model



Candidate Entities

Company

Represents an agency or independent-owner workspace.

Candidate fields:

- `id`
- `name`
- `status`
- `timezone`
- `default_currency`
- `created_at` , `updated_at`

AppUser

Represents a manager or staff member.

Candidate fields:

- `id` , `company_id`
- `name` , `email`
- `password_hash`
- `role` : manager or staff
- `status`
- `last_login_at`
- audit timestamps

The accepted MVP baseline uses globally unique app-user email addresses, as recorded in [docs/decisions/0001-initial-database-schema.md](#) . This simplifies login because a user does not need to select a company before authentication. If one person must later belong to several companies, the team will review this choice and introduce the required account-membership migration.

AuthSession

Represents one revocable browser session. It stores the company and user identity, a unique hash of the opaque token, a hash of the CSRF token, expiry, revocation, and audit timestamps. Raw session and CSRF secrets are never stored.

UserInvitation

Represents one manager-created staff invitation. It stores the company, invited user, inviter, a unique hash of the one-time token, expiry, acceptance, and audit timestamps. The development adapter may display the activation URL once; a production adapter must deliver it through an approved email provider.

Property

Represents one managed accommodation.

Candidate fields:

- `id` , `company_id`
- name and address fields
- city and timezone

- capacity and operational status
- check-in/out information
- Wi-Fi, parking, amenities, house rules, emergency contact
- external booking link
- archived timestamp
- audit timestamps

Sensitive operational fields require authorization and careful chatbot exposure.

Channel

Represents one property connection to one booking source.

Candidate fields:

- `id`, `property_id`
- platform or source type
- external listing identifier
- calendar URL or encrypted integration reference
- active state
- last successful synchronization time
- last error summary

Booking

Represents an imported or manually created reservation/block.

Candidate fields:

- `id`, `property_id`
- nullable `channel_id` for direct/manual records
- source type
- external event identifier
- guest name and contact when available
- check-in and check-out timestamps/dates
- status
- record type: reservation or blocked period
- last external update time
- audit timestamps

Candidate integrity rules:

- check-out must be after check-in;
- external identity should be unique within a channel;
- cancelled records do not count as occupied;
- adjacent stays are not overlaps when one checkout equals the next check-in;
- external payload may be stored in a controlled JSON field for troubleshooting, with retention review.

CalendarSyncRun

Records synchronization observability:

- channel
- start and completion time
- status
- created, updated, cancelled, and rejected event counts
- sanitized error information
- correlation identifier

BookingConflict

Represents detected overlap state rather than only a transient notification.

Candidate fields:

- property
- participating bookings, through association rows if needed
- detected time
- status: open, acknowledged, resolved, dismissed
- resolution note
- actor and resolution time

Conversation

Represents one guest communication context.

Candidate fields:

- `id`, `company_id`, `property_id`
- optional active/relevant `booking_id`
- guest contact identifier
- status
- handling mode: automatic or manual
- assigned staff user, optional
- last message time
- audit timestamps

Conversation identity needs testing. A practical MVP rule is one open conversation per company, property, and guest contact, with optional booking context.

Message

Candidate fields:

- `id`, `conversation_id`
- external provider/message identifier
- direction: inbound or outbound

- sender type: guest, chatbot, staff, system
- sender user ID where applicable
- language
- content
- delivery status
- automatic-send flag
- model/configuration reference where applicable
- confidence/escalation reason where applicable
- provider timestamps and local audit timestamps

Contractor

Represents a contact, not an authenticated user.

Candidate fields:

- `id`, `company_id`
- name
- phone
- specialty
- notes
- active state
- audit timestamps

Ticket

Candidate fields:

- `id`, `company_id`, `property_id`
- optional `booking_id` and `conversation_id`
- title and description
- category and priority
- status
- created by user
- suggested by chatbot flag/reference
- resolved time
- audit timestamps

TicketAssignment

Preserves assignment history:

- ticket and contractor
- assigned by user
- assigned time
- ended/reassigned time

- notes

Tenant-Isolation Strategy

The accepted MVP baseline stores `company_id` on every tenant-owned table and includes it in composite foreign keys between tenant-owned records, as recorded in `docs/decisions/0001-initial-database-schema.md`. This makes tenant filtering explicit and lets PostgreSQL reject relationships that cross company boundaries.

Application repositories and services must still derive the company from the authenticated server-side context and scope every query by that company. The database constraints provide defense in depth; they do not replace application authorization. Cross-company persistence tests are required before dependent feature modules are considered complete.

Data Lifecycle

- Prefer archive/deactivation over deleting operational history.
- Define retention for guest contacts, message content, external payloads, and logs.
- Provide correction and deletion procedures where legally required.
- Do not place secrets or access codes in logs.
- Back up production/pilot data and verify restore.

Migration Strategy

The first migration should create only approved entities and constraints. Every later schema change should use a reviewed Alembic migration. Autogenerated migrations must be inspected before execution because generation does not replace domain review [REF-ALEMBIC].

Database Review Checklist

- ☒ Final entity names approved for the initial MVP baseline
- ☒ Tenant-isolation strategy approved
- ☒ Email uniqueness decision approved
- ☒ Booking date and timezone semantics approved
- ☒ iCalendar cancellation policy approved
- ☒ Conversation identity rule approved
- ☒ Ticket status transitions approved
- ☒ Unique constraints and indexes identified
- ☒ Audit fields consistent for the initial MVP baseline
- ☒ Sensitive-data and retention policy documented
- ☒ Initial migration upgrade, downgrade, and re-upgrade run against clean PostgreSQL
- ☒ Cross-company foreign-key and booking date-range integrity checks pass

CHAPTER 11 · TECHNICAL SPECIFICATION

Chatbot and Integration Conception

Status: Proposed safety and integration baseline

Chatbot Role

The Vayca chatbot is a controlled communication capability, not an autonomous property manager. It may retrieve verified information, generate multilingual replies, classify requests, and suggest structured actions. It must not independently cancel bookings, resolve conflicts, assign contractors, approve refunds, or create high-impact operational changes.

Supported MVP Languages

- French
- English
- Arabic

Evaluation should include Modern Standard Arabic and representative Tunisian Arabic expressions. Broad model language support is not accepted as proof of local quality.

Response Policy

Automatic-answer category

- Wi-Fi details
- check-in and checkout times
- directions and parking
- amenities
- house rules
- approved emergency contact information
- availability obtained from the backend
- owner-configured external booking link

Human-confirmation category

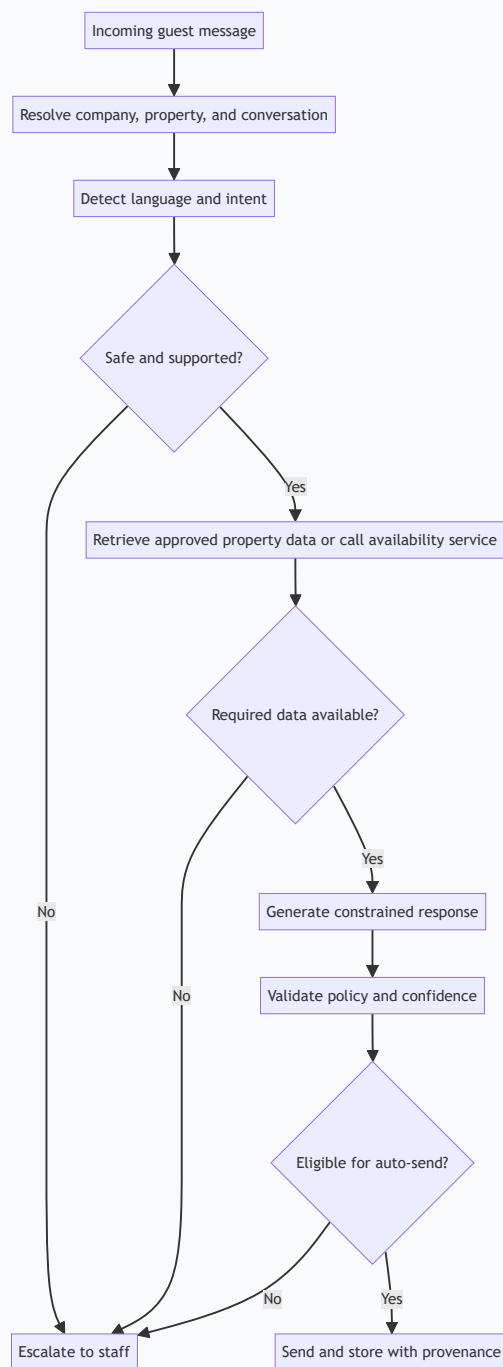
- proposed maintenance ticket
- proposed priority or issue category
- guest update after ticket change
- ambiguous special request

Mandatory-escalation category

- refund or payment dispute
- booking cancellation or date modification

- complaint
- emergency or safety issue
- booking conflict
- missing or contradictory property information
- low-confidence interpretation
- unsupported language or provider failure

Grounded-Answer Flow



Tool Boundary

The chatbot may request backend tools with structured input. The backend owns execution and authorization.

Candidate tools:

- `get_property_information(property_id, fields)`
- `check_availability(property_id, check_in, check_out)`
- `get_booking_context(conversation_id)`
- `suggest_ticket(conversation_id, category, priority, description)`

The `suggest_ticket` tool creates a suggestion, not a real ticket. Staff confirmation calls the normal ticket-creation use case.

Prompt and Model Strategy

- Keep model/provider selection configurable.
- Use a system policy that defines allowed, confirmation, and escalation actions.
- Provide only the minimum authorized property context.
- Separate deterministic backend results from generated natural language.
- Store model/configuration version for audit where feasible.
- Never include credentials or unrelated company data in prompts.
- Use evaluation results to compare quality, latency, and cost.

OpenAI's Evals API supports repeatable evaluation datasets and graders; the project may use it or an equivalent local evaluation harness [REF-OPENAI-EVALS].

Calendar Integration

iCalendar baseline

RFC 5545 defines a data format for exchanging calendar information independent of a particular calendar service [REF-IETF-ICAL]. Vayca's initial integration should treat iCalendar as an imported availability source, not promise unrestricted official API functionality.

Synchronization behavior

1. Manager adds a feed to a property channel.
2. System validates and stores the configuration securely.
3. Scheduled worker downloads the feed.
4. Parser normalizes supported events.
5. Upsert logic creates, updates, or cancels records idempotently.
6. Conflict detection runs for changed active periods.
7. Synchronization health is recorded and shown.

Limitations to communicate

- Feed freshness depends on polling and source behavior.

- iCalendar may not include complete guest, pricing, or reservation metadata.
- Importing availability is not equivalent to official two-way channel management.
- Vayca does not automatically alter source-platform reservations in the MVP.

WhatsApp Integration

Development modes

Mode	Purpose
Simulator	Deterministic local and CI testing
Provider test	Real webhook and message-delivery verification using test credentials/number
Production/pilot	Approved business account and real guest data

Webhook requirements

- Verify provider authenticity according to provider documentation.
- Acknowledge quickly and defer slow work.
- Deduplicate by external message/event identifier.
- Preserve delivery status changes.
- Avoid logging full sensitive payloads.
- Retry outgoing messages with bounded policy.
- Keep staff handling usable during provider or chatbot outage.

Implemented local simulator contract

The MVP development simulator accepts a signed event at `POST /api/v1/integrations/whatsapp/simulator/inbound`. The payload contains a property ID, guest contact identifier, message content, external message ID, and optional timestamp/language. It does not accept a company ID; the backend resolves company context from the stored property. The caller calculates an HMAC-SHA256 signature over the raw JSON request body using the local simulator secret and sends it in `X-Vayca-Simulator-Signature`.

The endpoint returns a `202` acknowledgement after idempotent persistence. It is a development/CI transport only and must not be described as an approved WhatsApp connection. Provider test and production modes remain separate future adapters.

Interactive WhatsApp Web guest simulator & presentation QR code

To enable realistic live demonstrations during academic defenses and jury evaluations without requiring external third-party provider accounts (Twilio / Meta WhatsApp Business), the platform provides an unauthenticated mobile guest simulator (`/simulator/whatsapp`) and workstation QR presentation modal:

1. **Presentation QR Modal (`SimulatorQrModal`)**: Accessible via "Démo WhatsApp" in the sticky workstation header, displaying a dynamic vector QR code generated via a zero-dependency ISO/IEC 18004 pure TypeScript engine (`qrCode.ts`). Evaluators scan the code with their smartphone camera to open the simulator directly.

2. **Guest Mobile Interface (`GuestWhatsAppSimulatorPage`)**: Replicates WhatsApp Web with green branding (`#075E54`), dynamic villa selection, configurable guest telephone number (persisted in `localStorage`), message bubbles with origin badges ( IA Vayca vs  Équipe Vayca), typing animation, and 1-tap quick prompts (*WiFi, Piscine, Check-out, Panne Clim*).
3. **Guarded API Endpoints**: Dedicated endpoints (`/api/v1/integrations/whatsapp/simulator/chat/*`) allow fetching active properties, querying chat message history, and dispatching inbound messages into Celery's `process_chatbot_inbound_message` pipeline. All endpoints are guarded by `check_simulator_mode()` , disabling them automatically when production adapters are configured.

Ticket Suggestion Example

Guest message:

Water is leaking under the kitchen sink.

Chatbot result:

```
{
  "action": "suggest_ticket",
  "category": "plumbing",
  "priority": "medium",
  "description": "Guest reports water leaking under the kitchen sink.",
  "requires_staff_confirmation": true
}
```

The interface shows the suggestion. Staff may change priority, add context, select a contractor, and confirm.

Evaluation Dataset

The team should maintain versioned examples for:

- common property questions;
- unavailable knowledge;
- available and unavailable date ranges;
- date ambiguity;
- complaints and refunds;
- maintenance descriptions;
- emergencies;
- prompt-injection attempts;
- French, English, Modern Standard Arabic, and Tunisian Arabic;
- messages belonging to the wrong or unknown property.

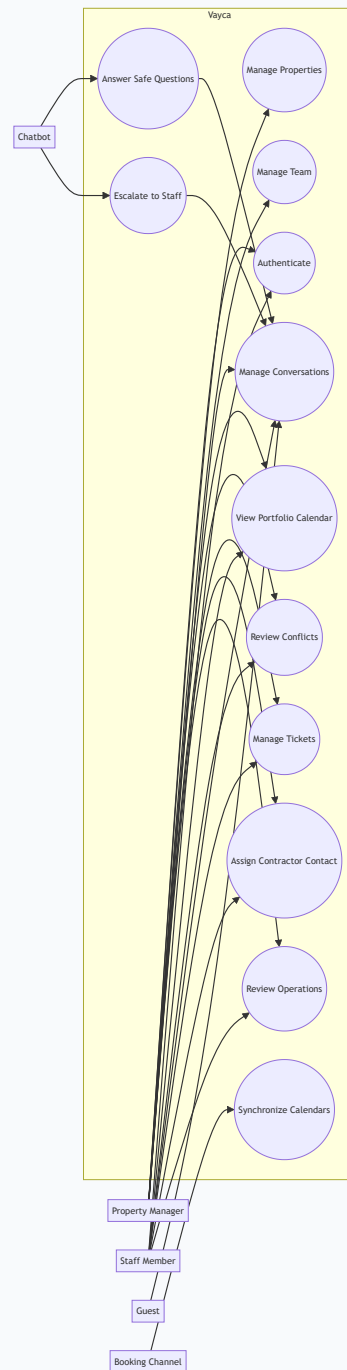
Metrics should include correctness, groundedness, correct escalation, language appropriateness, latency, and estimated cost.

CHAPTER 12 · TECHNICAL SPECIFICATION

UML and Workflow Models

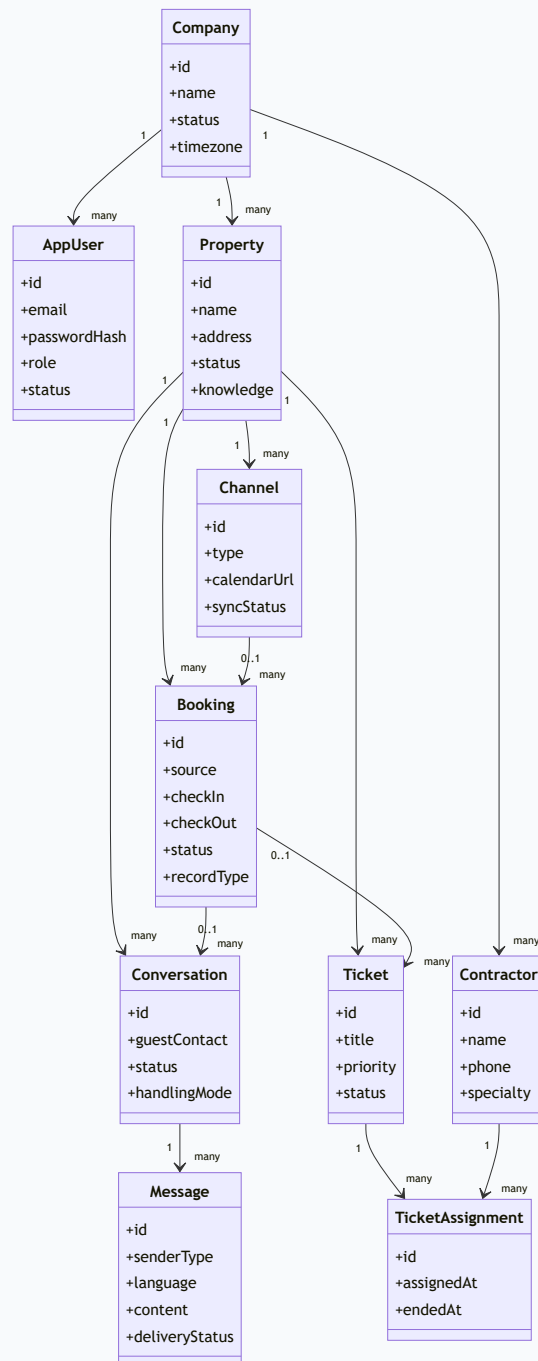
Status: Editable conception models; diagrams must evolve with approved requirements

System Use-Case Diagram

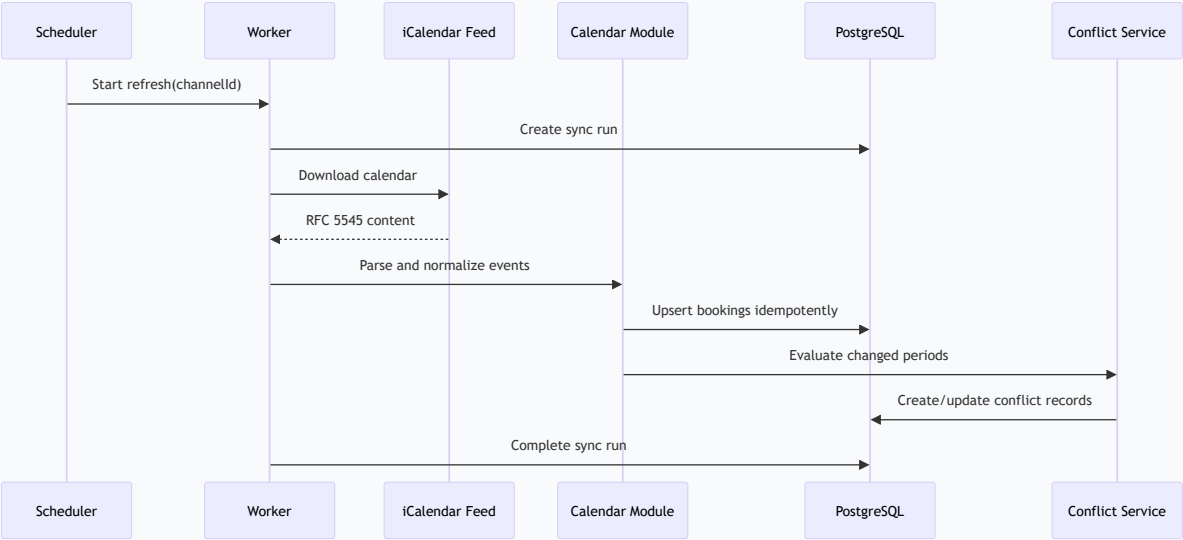


The contractor is intentionally absent as a system actor because contractors do not authenticate in the MVP.

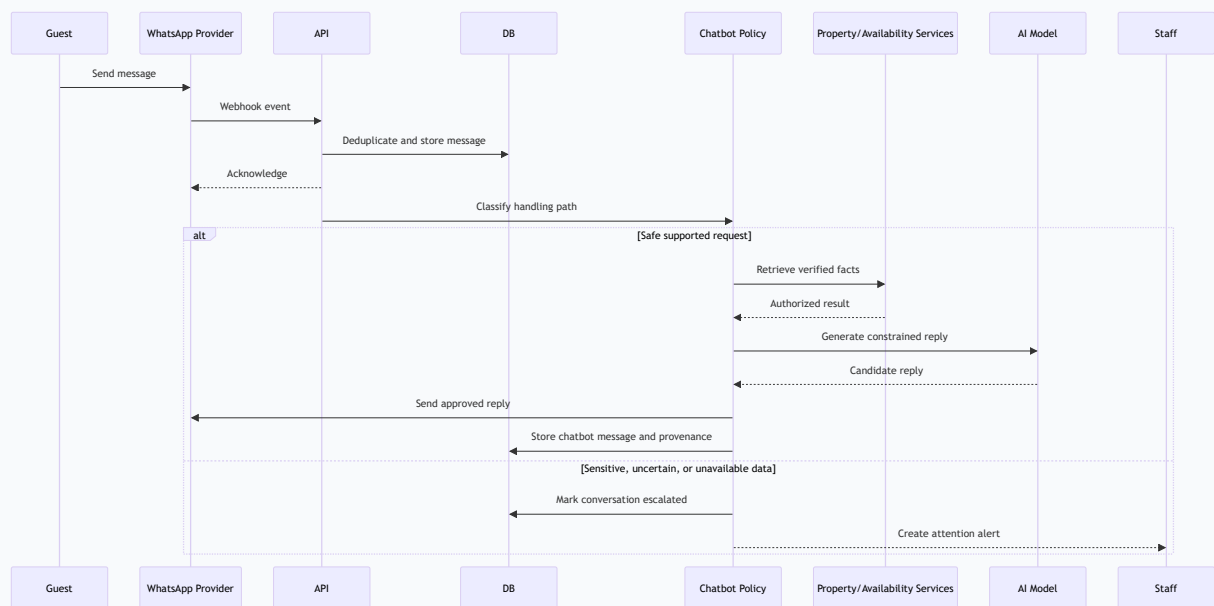
Domain Class Diagram



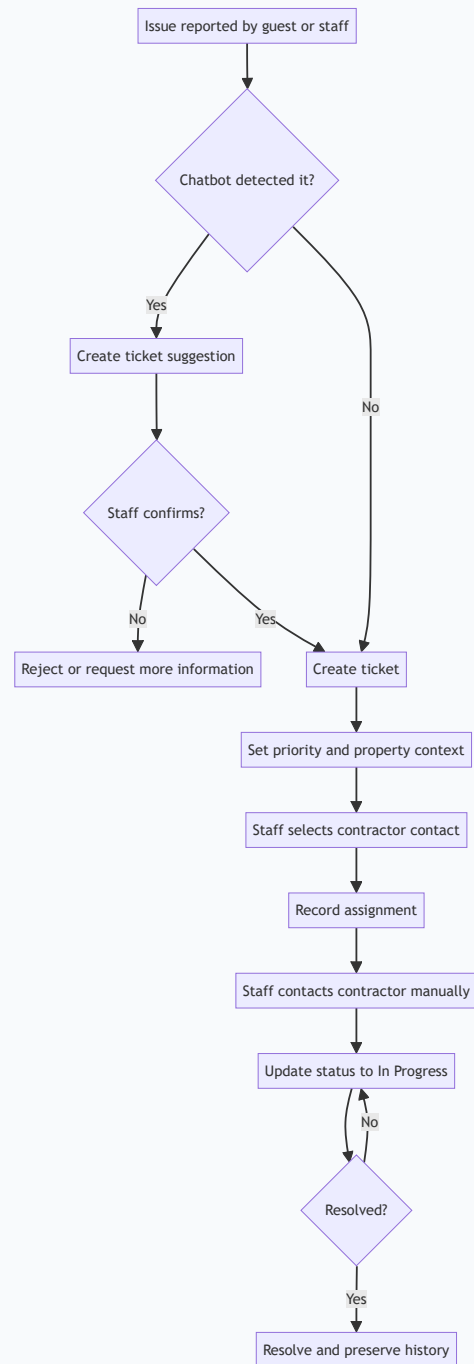
Calendar Synchronization Sequence



Guest Message and Chatbot Sequence



Maintenance Activity Diagram



Deployment Diagram

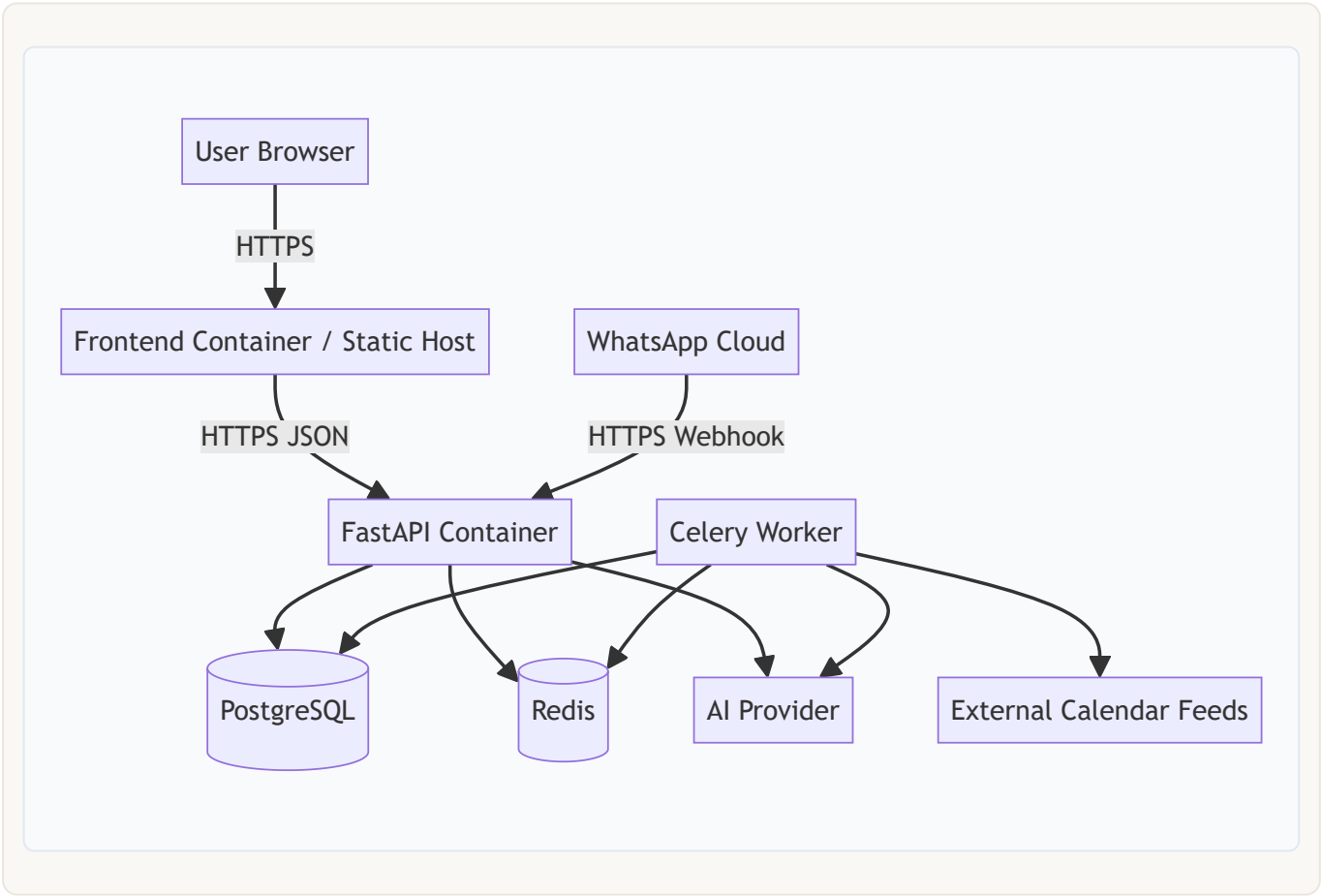


Diagram Maintenance Rule

When an approved requirement changes an actor, entity, workflow, module dependency, or deployment component, the related diagram must be updated in the same pull request.

CHAPTER 13 · TECHNICAL SPECIFICATION

Project Management and Team Organization

Status: Proposed working method; team allocation requires team confirmation

Team Context

Vayca is developed by a three-person internship team. The team should use a lightweight iterative method with explicit requirements and reviewed integration. The objective is not to reproduce a large-company process. It is to make parallel work understandable, testable, and recoverable.

Recommended Method

Use short iterations with these stages:

1. Select an approved requirement and GitHub issue.
2. Confirm dependencies and acceptance criteria.
3. Create a feature branch from the current `testing` branch.
4. Implement one reviewable unit.
5. Run focused tests and document evidence.
6. Open a pull request into `testing`.
7. Obtain teammate review.
8. Integrate and verify the combined environment.
9. Promote stable `testing` changes to `main` for demonstration releases.

Branching Model

```
main
 ^
 | reviewed promotion
testing
 ^
 | reviewed feature pull requests
feature/<short-issue-name>
```

Rules:

- Never develop directly on `main` or `testing`.
- Create feature branches from current `testing`.
- Keep one issue or tightly related change per branch.
- Use `feat:`, `fix:`, `docs:`, `test:`, or `chore:` commit prefixes.
- Require at least one teammate review.
- Delete merged feature branches.

GitHub Project Structure

Recommended workflow statuses:

- Backlog
- Ready
- In Progress
- In Review
- Done
- Resources, for non-delivery reference material only

Recommended project fields:

- Phase: MVP or Phase 2
- Module: Foundation, Calendar, Communication, Maintenance, Supervision
- Priority: Must, Should, Could
- Owner
- Iteration
- Dependency or blocked-by reference
- Requirement IDs

Issue Quality Standard

Every implementation issue should include:

```
## Problem
Which approved user or system need is addressed?

## Scope
What is included and explicitly excluded?

## Requirements
FR-... / NFR-...

## Dependencies
Blocked by #...

## Acceptance criteria
- [ ] Observable result
- [ ] Failure/empty state
- [ ] Authorization behavior
- [ ] Test evidence

## Documentation impact
Files or diagrams that must change
```

Large backlog items such as “WhatsApp integration” should be split into webhook verification, inbound persistence, conversation resolution, outgoing delivery, delivery-status handling, and simulator/test-mode work.

Suggested Work Packages

Iteration 0: Conception freeze

- Approve needs, scope, actors, navigation, modules, and provisional domain model.
- Reconcile GitHub issues with requirement IDs.
- Agree API and error conventions.
- Verify clean Docker startup.

Iteration 1: Foundation and properties

- Database migration environment
- Company and user model
- Authentication and authorization
- Property API and connected interface

Iteration 2: Calendar core

- Manual bookings
- Availability service
- iCalendar parser and synchronization records
- Conflict detection
- Connected calendar interface

Iteration 3: Communication core

- Conversation and message model
- Webhook simulator
- Inbox and conversation interface connected to API
- Manual takeover

Iteration 4: Chatbot and maintenance

- Grounded property answers
- Availability tool
- Escalation evaluation
- Ticket suggestions and confirmation
- Contractor contacts and ticket lifecycle

Iteration 5: External integration and supervision

- WhatsApp test integration
- Integration failure handling
- Dashboard read models
- Security, performance, and mobile validation

Iteration 6: Pilot and academic finalization

- End-to-end demonstration
- Pilot feedback
- Defect correction
- Final diagrams, screenshots, test results, and report generation

Definition of Ready

An issue is ready when:

- requirement IDs are known;
- acceptance criteria are observable;
- dependencies are complete or planned;
- API/data decisions are sufficient;
- responsible member is identified;
- no unresolved team-level architectural decision blocks it.

Definition of Done

An issue is done when:

- implementation is connected to persistent or intentionally simulated data;
- authorization is enforced where applicable;
- normal, empty, loading, and failure behavior is handled;
- focused tests pass;
- relevant documentation is updated;
- a teammate reviewed the pull request;
- behavior is verified after merge into `testing`.

Documentation Responsibilities

Documentation is shared work even if one person coordinates it. Each module owner updates:

- requirements affected by changes;
- API and data decisions;
- relevant UML/workflow diagrams;
- test evidence;
- risks or limitations discovered.

The documentation coordinator ensures structure and consistency but should not invent module decisions without review.

CHAPTER 14 · TECHNICAL SPECIFICATION

Testing and Validation Strategy

Status: Implemented baseline and active validation record

Objectives

- Verify that each requirement has observable evidence.
- Protect critical rules such as tenant isolation, date overlap, and chatbot escalation.
- Detect integration failures before pilot use.
- Validate usability with target users, not only developers.
- Distinguish prototype completeness from functional correctness.

Verified Test Suite Summary (Production Baseline v1.1.0)

Suite	Scope / Tool	Test count	Result
Backend Domain & Auth	Pytest (test_api_conventions.py , test_authentication.py , test_database_schema.py)	26 passed	Verified in Docker
Properties & Calendar	Pytest (test_properties.py , test_calendar.py , test_manual_bookings.py)	30 passed	Verified in Docker
Receipts & Billing	Pytest (test_booking_receipts.py)	6 passed	Verified in Docker
Messaging & Chatbot	Pytest (test_messaging.py , test_chatbot_policy.py , test_chatbot_grounding.py)	20 passed	Verified in Docker
Communication Adapters	Pytest (test_email_adapter.py , test_whatsapp_adapter.py , test_whatsapp_simulator_chat.py)	26 passed	Verified in Docker
Maintenance & Tickets	Pytest (test_maintenance.py)	6 passed	Verified in Docker
Supervision & Statements	Pytest (test_supervision.py , test_owner_payout_statements.py)	14 passed	Verified in Docker
Post-Stay Reviews	Pytest (test_review_sequences.py)	7 passed	Verified in Docker
Cross-Module MVP Scenario	Pytest (test_mvp_validation.py)	1 passed	Verified in Docker
Total Backend	Pytest / Docker Compose backend_test profile	136 passed (18 modules)	100% Green
Frontend Suites	Vitest + Testing Library (26 test suites including PWA, Owner Portal, Simulator)	132 passed	100% Green
Frontend Static Checks	Oxlint linter & TypeScript build (tsc -b && vite build)	0 errors	Clean build
CI / CD Workflows	GitHub Actions (backend-tests , frontend-tests , stack-smoke)	3 jobs	Passing

Test Levels

Unit tests

Focus on deterministic domain rules:

- booking-overlap calculation;
- availability calculation;
- ticket status transitions;
- role-permission decisions;
- calendar-event normalization;
- chatbot policy classification helpers.

Integration tests

Verify collaboration with real infrastructure or controlled substitutes:

- API with PostgreSQL;
- Alembic migration upgrade from a clean database;
- Redis/Celery task dispatch;
- iCalendar fixture import;
- webhook deduplication;
- outgoing provider adapter with a fake server;
- tenant-isolation queries.

API contract tests

- request and response validation;
- correct status codes;
- authentication and authorization;
- pagination and filtering;
- stable error format;
- OpenAPI schema generation.

Frontend tests

- key component behavior;
- route protection;
- loading, empty, failure, and success states;
- API error display;
- mobile navigation;
- form validation;
- status labels and accessibility.

End-to-end tests

Critical scenarios:

1. Manager logs in, creates a property, and adds operational knowledge.
2. Calendar import creates bookings and displays them.
3. Overlapping booking produces a visible conflict alert.
4. Manual booking changes availability.
5. Guest simulator message creates a conversation.
6. Safe question receives a grounded response.
7. Sensitive question escalates and staff takes over.
8. Maintenance report becomes a staff-confirmed ticket.
9. Staff assigns a contractor contact and resolves the ticket.
10. Dashboard reflects the resulting live data.

Critical Test Matrix

Area	Normal case	Edge/failure case
Authentication	Valid manager/staff login	Invalid, inactive, expired, rate-limited
Tenant isolation	User reads own company data	User attempts another company's IDs
Calendar import	Valid event create/update	Invalid feed, duplicate event, outage, cancellation
Overlap	True overlapping periods	Adjacent stays, cancelled booking, different property
Availability	Free and occupied ranges	Invalid range, partial overlap, unknown property
Webhook	New incoming message	Duplicate, invalid signature, unknown property/contact
Chatbot	Known safe answer	Missing fact, complaint, emergency, provider outage
Ticket	Valid create and transition	Invalid transition, unauthorized update, archived property
Dashboard	Correct derived counts	Empty portfolio and stale integration

Chatbot Evaluation

Maintain a versioned dataset with expected outcomes. Each example includes:

- language;
- property/booking context;
- guest message;
- expected intent;
- expected allowed action;
- expected facts;
- whether escalation is mandatory;
- quality notes.

Initial quality gates

- No fabricated availability in the evaluation set.
- No automatic answer to defined mandatory-escalation categories.

- Property facts match the supplied context.
- Responses remain understandable in supported languages.
- Provider failure leaves the conversation available to staff.

These gates should become numeric after the dataset is sufficiently large. Ten questions are useful for initial debugging but insufficient evidence for production reliability.

Usability Validation

Conduct task-based sessions with representative users. Suggested tasks:

- find today's arrivals;
- identify a booking conflict;
- answer an escalated guest message;
- create and assign a maintenance ticket;
- update property check-in instructions.

Record:

- task completion;
- completion time;
- errors and hesitation;
- terms users do not understand;
- assistance required;
- qualitative feedback.

Security Verification

Use a manageable subset of OWASP ASVS as guidance [REF-OWASP-ASVS]. At minimum verify:

- authentication and session handling;
- authorization and tenant isolation;
- input validation;
- password and secret storage;
- logging without sensitive-data leakage;
- webhook authenticity and replay resistance;
- dependency and container vulnerability review before pilot deployment.

Test Environments and Data

- Use synthetic guest and booking data in development and CI.
- Keep test credentials separate from production credentials.
- Do not use real guest conversations in automated tests without authorization and anonymization.
- Reset test databases deterministically.
- Store iCalendar and webhook fixtures in version control without secrets.

Acceptance Evidence

Each completed issue should record:

- tests executed;
- result observed;
- environment used;
- relevant screenshots for UI work;
- known limitations;
- requirement IDs verified.

CHAPTER 15 · TECHNICAL SPECIFICATION

Risks, Constraints, and Mitigation

Status: Active risk register

Risk Scale

Probability and impact are rated Low, Medium, or High. Priority is derived qualitatively and should be reviewed each iteration.

Risk Register

ID	Risk	Probability	Impact	Mitigation	Owner
R-01	MVP scope exceeds three-person capacity	High	High	Keep explicit exclusions; require trade-off for additions	Team lead
R-02	Official booking APIs are unavailable	High	Medium	Use RFC 5545 calendar feeds and manual bookings; label limitations	Calendar lead
R-03	Calendar feed is stale, incomplete, or unavailable	Medium	High	Polling status, retries, last-known data, visible errors, manual refresh	Calendar lead
R-04	Chatbot sends an incorrect or harmful answer	Medium	High	Grounding, safe categories, mandatory escalation, evaluations, audit	Communication lead
R-05	Tunisian Arabic quality is inadequate	Medium	Medium	Local test set, human review, escalation, phased language tuning	Communication lead
R-06	WhatsApp approval is delayed	Medium	High	Simulator and provider test mode; integration adapter	Communication lead
R-07	One company accesses another company's data	Low/Medium	Critical	Central tenant scoping, authorization tests, code review	Platform lead
R-08	Team branches implement incompatible contracts	Medium	High	OpenAPI contracts, small issues, review, frequent integration	Entire team
R-09	Database schema is implemented before workflow approval	Medium	High	Team review and provisional conception checklist before migration	Database lead
R-10	Prototype UI is mistaken for completed functionality	Medium	Medium	Truthful statuses, connected-data criteria, definition of done	Entire team
R-11	Docker setup differs between teammates	Medium	Medium	Clean-clone startup test and pinned dependencies	Platform lead
R-12	Sensitive guest/property information leaks through logs or prompts	Medium	High	Data minimization, sanitized logs, authorization, retention policy	Platform and AI leads

ID	Risk	Probability	Impact	Mitigation	Owner
R-13	Lack of direct customer interviews weakening study confidence	Medium	Medium	Validate workflows against documented domain requirements and automated testing; field interviews tracked as an open backlog issue (#34) for post-MVP pilots	Documentation coordinator
R-14	Documentation and implementation drift	High	Medium	Requirement IDs, traceability, documentation in same PR	Entire team
R-15	Dependency or provider cost becomes excessive	Medium	Medium	Usage limits, model evaluation, configurable provider/model	Team lead
R-16	Demo depends on unstable external services	Medium	High	Deterministic fixtures and simulation mode	Entire team

Technical Constraints

- PostgreSQL is the persistent database baseline.
- FastAPI and React/TypeScript are the application baseline.
- Docker is the shared development environment.
- Calendar support begins with supported iCalendar data.
- Guests and contractors do not authenticate.
- External providers may impose approval, rate, content, and delivery restrictions.

Organizational Constraints

- Three members must divide development and academic work.
- Members have limited prior group-development experience.
- Supervisor review may require changes to study or conception.
- Internship deadlines limit integration breadth and pilot duration.

Ethical and Legal Considerations

- Guests should not be misled into believing uncertain chatbot answers are verified facts.
- Automated messages should be attributable and reviewable.
- Personal data should be collected only for defined operational purposes.
- Real data use should follow Tunisian personal-data obligations and provider terms.
- The final report should clearly distinguish evidence, assumptions, simulation, and implemented functionality.

Contingency Demonstration

The project must support an end-to-end demonstration with controlled fixtures if external services are unavailable:

- iCalendar fixture server or local feed;
- WhatsApp webhook simulator;
- fake outgoing-message adapter;
- configurable chatbot stub for deterministic escalation cases;

- seeded multi-tenant demonstration database.

This fallback is not a substitute for integration work. It prevents the academic demonstration from depending entirely on external service availability.

CHAPTER 16 · TECHNICAL SPECIFICATION

Requirements Traceability Matrix

Status: Active baseline (MVP implementation and system specification fully verified)

Purpose

Traceability connects stakeholder needs to requirements, modules, implementation work, and verification. All MVP functional issues are implemented and covered by automated test suites. Empirical customer field interviews are tracked as an open backlog item (Issue #34) for post-MVP deployment.

Functional Traceability

Need	Requirements	Module	Current GitHub issue	Primary verification
Shared environment and contracts	NFR-MNT-03, NFR-DEP-01..04	Foundation	#30	Clean-start and contract verification
Secure company access	FR-AUTH-01..06	Foundation	#18, #19	Auth, role, and migration tests
Manage property information	FR-PROP-01..05	Properties/Calendar	#7	CRUD and tenant-isolation tests
Connect calendar source	FR-CAL-01..03, FR-CAL-09	Properties/Calendar	#3, #4	Feed fixture and idempotency tests
Record direct reservations and calculate availability	FR-CAL-04, FR-MSG-07	Properties/Calendar	#31	Manual-booking and availability tests
Configure and approve direct-booking prices	FR-CAL-10, NFR-REL-04..05	Properties/Calendar	#21	Pricing-profile, quote, override-audit, and tenant-isolation tests
Detect booking conflicts	FR-CAL-05, FR-CAL-08	Properties/Calendar	#5	Date-overlap matrix
View portfolio calendar	FR-CAL-06..07	Properties/Calendar	#6	Connected UI and mobile test
Receive guest messages	FR-MSG-01..04	Communication	#8, #11, #12	Webhook, deduplication, and UI tests
Answer in guest language	FR-MSG-05..07	Communication/Chatbot	#9	Multilingual grounded evaluation
Escalate and take over	FR-MSG-08..10	Communication/Chatbot	#10, #11	Sensitive-message and takeover tests
Suggest ticket from issue	FR-MSG-11..12	Communication/Maintenance	#13	Suggest/confirm/reject scenarios
Create maintenance ticket	FR-TKT-01..02	Maintenance	#13	Ticket validation tests

Need	Requirements	Module	Current GitHub issue	Primary verification
Manage contractors and assignment	FR-TKT-03..04	Maintenance	#14	Contact and assignment-history tests
Track status and history	FR-TKT-05..07	Maintenance	#15	Transition and history tests
Update guest about issue	FR-TKT-08	Maintenance/Communication	#16	Approved-message test
Review daily operations	FR-DASH-01..04	Supervision	#17	Derived-data and navigation tests
Manage settings and integration health	FR-SET-01..04	Foundation/Supervision	#18, #32	Role and truthful-status tests
Validate the integrated MVP	Cross-cutting NFRs	Cross-cutting	#33	Staging end-to-end evidence (<code>test_mvp_validation.py</code>)
Post-MVP field validation and customer discovery	Post-MVP pilots	Documentation / Roadmap	#34 (Open)	Open backlog issue for operational field pilot interviews

Non-Functional Traceability

Quality concern	Requirements	Design response	Verification
Authentication and authorization	NFR-SEC-01, 03, 05, 07	Argon2id, revocable opaque sessions, CSRF, Redis rate limiting, HTTPS	Security tests and deployment review
Tenant isolation	NFR-SEC-02, 06	Company-scoped repositories and role checks	Cross-company tests
Secret protection	NFR-SEC-04	Environment configuration and Git exclusions	Secret scan
Idempotency and recovery	NFR-REL-01..04	External IDs, sync runs, transactions, retries	Duplicate and outage tests
Auditability	NFR-REL-05, NFR-AI-03	Actor timestamps and chatbot provenance	Database verification
Performance	NFR-PERF-01..04	Async jobs, indexed queries, load fixtures	API and worker load tests
Usability	NFR-UX-01..06	Six-section navigation and explicit states	Task-based user tests
Maintainability	NFR-MNT-01..06	Modules, migrations, OpenAPI, small PRs	Review and CI evidence
Deployment	NFR-DEP-01..04	Docker environments and health checks	Clean-clone/staging test
Chatbot safety	NFR-AI-01..06	Tool grounding, policy, escalation, provider fallback	Versioned evaluation set

Backlog Reconciliation Actions

1. Add a Foundation epic or grouping for company, authentication, migrations, and shared API conventions.
2. Split property CRUD from property knowledge if one issue becomes too large.
3. Split each calendar provider/feed issue into configuration, import, update/cancellation, and health visibility if needed.

4. Add manual/direct booking and availability-service issues.
5. Split WhatsApp transport from conversation persistence.
6. Separate chatbot safe answers, availability tool, escalation, and evaluation dataset.
7. Add ticket-suggestion confirmation as explicit work.
8. Correct frontend issue statuses only after deciding whether prototype completion counts as design/prototype work or implementation.
9. Add requirement IDs and dependency links to every MVP issue.
10. Keep Phase 2 issues outside active MVP views.

Change Traceability

When a requirement changes:

1. Update its requirement text and status.
2. Update affected use cases and diagrams.
3. Update the data or architecture chapter if applicable.
4. Update GitHub issues and acceptance criteria.
5. Update test cases.
6. Add a decision-log entry for material changes.

CHAPTER 17 · TECHNICAL SPECIFICATION

References, Glossary, and Decision Log

Status: Active supporting chapter

References

Tunisian sector and legal context

- **REF-INS-CST-2024:** Institut National de la Statistique, Tunisia, "Compte Satellite du Tourisme 2023-2024," 2026. <https://www.ins.tn/publication/compte-satellite-du-tourisme-2023-2024>
- **REF-INS-ACCOMMODATION:** Institut National de la Statistique, Tunisia, "Nuitées touristiques et hébergement" statistical series. <https://www.ins.tn/statistiques/130>
- **REF-INPDP-LAW:** Instance Nationale de Protection des Données Personnelles, Organic Law No. 2004-63 of 27 July 2004 concerning personal-data protection. https://www.inpdp.tn/ressources/loi_2004.pdf
- **REF-INPDP-FORMS:** INPDP, declaration and authorization procedures for personal-data processing. <https://www.inpdp.tn/Formulaires.html>

Technical standards and official documentation

- **REF-IETF-ICAL:** Internet Engineering Task Force, RFC 5545, "Internet Calendaring and Scheduling Core Object Specification (iCalendar)." <https://datatracker.ietf.org/doc/html/rfc5545>
- **REF-FASTAPI-SECURITY:** FastAPI documentation, "OAuth2 with Password (and hashing), Bearer with JWT tokens." <https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>
- **REF-ALEMBIC:** SQLAlchemy project, Alembic documentation. <https://alembic.sqlalchemy.org/en/latest/>
- **REF-OWASP-ASVS:** OWASP Foundation, Application Security Verification Standard. <https://owasp.org/www-project-application-security-verification-standard/>
- **REF-OWASP-SESSION:** OWASP Foundation, Session Management Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- **REF-OPENAI-EVALS:** OpenAI API documentation, Evals. <https://platform.openai.com/docs/api-reference/evals>

Project sources

- Root `README.md`
- `AGENTS.md`
- `docs/architecture.md`
- `docs/design-system.md`
- `design_system.md`
- `VacayOps_Project_Report.pdf`
- `VacayOps_Conception_and_Database_Design.pdf`
- GitHub repository issues and Project "app features"

Citation Policy

- Prefer official public sources for statistics, standards, laws, and technical behavior.
- Record access dates during final PDF production if required by the institution.
- Do not repeat unsupported market-size or competitor claims from earlier reports without verification.
- Mark interview evidence with anonymized participant identifiers and interview dates.
- Distinguish project decisions from externally verified facts.

Glossary

Term	Meaning in Vayca
Agency	Company managing properties for itself or owners
Independent owner	Small Vayca customer represented by a normal company workspace
Availability	Whether a property has no active blocking record in a requested period
Booking	Imported or manually entered reservation record
Blocked period	Calendar period unavailable without necessarily containing full reservation data
Channel	A property's connection to an external booking source
Chatbot	Controlled AI-supported guest communication capability
Company	Tenant boundary containing users, properties, contractors, and operational data
Conflict	Relevant overlap between active booking/block records for one property
Contractor	External maintenance contact without a Vayca account
Escalation	Transfer of a conversation to staff attention
Grounding	Restricting chatbot answers to verified context and backend results
iCalendar/iCal	Standard calendar exchange format defined by RFC 5545
Idempotency	Property that repeating the same event does not create duplicate effects
Manager	Authenticated user with company-administration permission
Marketplace	Public searchable booking platform; excluded from Vayca MVP
Multi-tenancy	One system serving several isolated company workspaces
Staff	Authenticated operational user without manager-only administration access
Ticket	Structured maintenance or operational issue record
Webhook	Provider-initiated HTTP event sent to Vayca

Decision Log

ID	Date	Status	Decision	Rationale / consequence
DEC-001	2026-07-28	Accepted	Vayca remains a B2B operations platform.	Avoids marketplace complexity and preserves original problem focus

ID	Date	Status	Decision	Rationale / consequence
DEC-002	2026-07-28	Accepted	Independent owners use the same company architecture and may receive a lower-priced plan later.	No second product or schema is required
DEC-003	2026-07-28	Rejected for MVP	Public marketplace and guest booking checkout.	Adds discovery, payments, cancellations, reviews, and public support obligations
DEC-004	2026-07-28	Accepted	Contractors do not authenticate.	Staff records assignment and contacts them manually
DEC-005	2026-07-28	Accepted	Main navigation uses Dashboard, Calendar, Messages, Maintenance, Properties, and Settings.	Reduces technical terminology and navigation complexity
DEC-006	2026-07-28	Accepted	iCalendar is the initial calendar integration candidate.	Accessible and sufficient for imported availability, with documented limitations
DEC-007	2026-07-28	Accepted	WhatsApp uses simulator, test, and later production modes.	Development remains possible during provider approval or outages
DEC-008	2026-07-28	Accepted	The user-facing term is chatbot.	Clearer for the target audience than AI assistant or autonomous agent
DEC-009	2026-07-28	Accepted	Chatbot answers safe questions, uses backend tools, and escalates sensitive/uncertain cases.	Prevents autonomous high-impact decisions
DEC-010	2026-07-28	Accepted	Ticket creation/assignment from chatbot output requires staff confirmation.	Preserves human responsibility
DEC-011	2026-07-28	Accepted	MVP chatbot languages are French, English, and Arabic, with Tunisian Arabic evaluation.	Matches current target while controlling scope
DEC-012	2026-07-28	Accepted	Manual/direct bookings are included without creating a marketplace.	Owners can block dates received through phone or WhatsApp
DEC-013	2026-07-30	Accepted	Authentication uses Argon2id passwords and revocable opaque server-backed sessions in secure cookies.	Avoids browser-stored credentials and supports immediate revocation
DEC-014	2026-07-28	Accepted	Academic documentation is written in English.	Team decision
DEC-015	2026-07-28	Accepted	These documents are a living baseline, not immutable final decisions.	Implementation and validation may reveal necessary changes
DEC-016	2026-08-22	Accepted	Grounded chatbot provider and deferred reply processing.	Policy and backend facts run before the configured provider; simulator remains deterministic and outbound delivery remains unconfigured.
DEC-017	2026-08-28	Accepted	Property detail fetches live calendar bookings.	Eliminates mock data dependencies on authenticated property screens and ensures truthful loading, empty, and retryable states.
DEC-018	2026-09-17	Accepted	Defer empirical field client interviews to post-MVP roadmap (tracked in open Issue #34).	Keeps MVP focused on verified software implementation and automated testing while tracking future field pilot interviews in the repository backlog.

Open Decisions

ID	Decision needed	Responsible discussion
OPEN-001	Resolved by Decision 0003: opaque session tokens, hashed server-side storage, and secure cookies	Closed 2026-07-30
OPEN-002	Global or company-scoped user email uniqueness	Database and UX review
OPEN-003	Final tenant-key strategy on child tables	Database review
OPEN-004	Exact booking date/time and timezone semantics	Calendar/database review
OPEN-005	iCalendar missing-event cancellation policy	Calendar integration testing
OPEN-006	Conversation identity and reopening rules	Messaging/database review
OPEN-007	Resolved by Decision 0009: forward-only ticket lifecycle and terminal assignment closure	Closed 2026-08-24
OPEN-008	AI provider and model	Decision 0008 selects the adapter approach; production model enablement remains subject to evaluation results
OPEN-009	Hosting platform and production topology	Deployment review
OPEN-010	Institution-specific report template and citation style	Supervisor confirmation

Finalization Checklist

- ☒ Post-MVP pilot and customer interview framework defined and tracked in open Issue #34
- ☒ Reconcile approved schema and migration with data conception
- ☒ Add screenshots only after connected implementation exists
- ☒ Add test results and measured performance
- ☒ Update GitHub issue statuses and requirement links
- ☒ Record all material scope decisions
- ☒ Confirm legal/privacy obligations for pilot data
- ☒ Apply institution formatting and citation requirements
- ☒ Render Mermaid diagrams at publication quality
- ☒ Generate and visually inspect the combined PDF

CHAPTER 18 · TECHNICAL SPECIFICATION

UI/UX Prototype Architecture & Interaction Specification

Status: Approved Architectural Specification & Senior Designer Baseline
Project: Vayca Executive Operations Workstation (3851962757848407279)
Design System Asset: assets/496e7233faec4294af96b3635c2df3d6 (Mediterranean Architectural Operations)
Target Viewports: Desktop (1440x900px / 2560x1440px), Mobile Phone (390x844px iPhone 15 Pro / 360px Android)
Primary Verification: Academic Traceability against NFR-UX-01 through NFR-UX-06 and NFR-AI-02

1. Executive Summary & Design System Foundations

The Vayca interface marries the structural serenity of Sidi Bou Said's Mediterranean vernacular with the high-density utility required by commercial hospitality operations. Designed for Tunisian property agencies and independent luxury villa operators, the platform replaces generic SaaS "card soup" with an **architectural split-pane workstation layout**.

1.1 Color Token Matrix & Semantic Roles

Token Name	Hex Value	Tailwind Class	Semantic Usage & Contrast Ratio
Sidi Bou Said Azure	#0F3D5E	bg-azure-600 , text-azure-600	Primary brand authority, left navigation icons, secondary actions, selected row borders. AAA contrast (9.8:1) against #FAF8F5 .
Azure Surface Highlight	#F0F6FA	bg-azure-50	Active navigation rail item background, highlighted table rows, selected chips.
Azure Sub-Border	#B6DAEA	border-azure-200	Active selection boundaries, secondary chip borders.
Terracotta Clay	#D96B43	bg-terracotta-500	Main transactional action accent, primary CTA buttons (+ New Booking , + New Order), urgent badges. AAA contrast (4.6:1) on white.
Terracotta Hover	#C25730	hover:bg-terracotta-600	Darkened state on hover/tap.
Terracotta Alert Surface	#FDF4F0	bg-terracotta-50	Alert strip background for escalations and warnings.
Terracotta Alert Stroke	#FBE6DC	border-terracotta-100	1px border framing escalation banners.
Warm Sun Amber	#E8A838 / #CF9024	bg-amber-500	AI contractor recommendation pills, star ratings, direct channel indicators.
Warm Sand Base	#FAF8F5	bg-sand-50	Fundamental app background. Replaces harsh clinical whites with an eye-resting lime-plaster daylight tone.
Pure White Surface	#FFFFFF	bg-white , bg-sand-100	Card surfaces, workstation panes, tabular containers. Defined by 1px hairline boundary.
Architectural Hairline	#EBE6DD	border-sand-200	1px structural hairline framing cards, panes, table rows, and split viewports. Avoids heavy drop shadows.

Token Name	Hex Value	Tailwind Class	Semantic Usage & Contrast Ratio
Dark Espresso Text	#1C1B18	text-sand-900	Primary headings, table row primary text, high-legibility numerals.
Muted Sand Text	#78716C	text-sand-500	Secondary metadata, timestamps, channel subtitles, helper text.
Critical Red	#DC2626	bg-red-600 , text-red-600	Calendar booking overlaps, conflict banners, emergency maintenance.
Conflict Surface	#FEF2F2 / #FECACA	bg-red-50 , border-red-200	Conflict alert banner background and border.
Success Green	#16A34A	bg-green-600 , text-green-600	Synced channel health dot, resolved work orders, confirmed payout badges.

1.2 Typography Hierarchy

Uniformly typeset in **Plus Jakarta Sans** (Google Fonts) with geometric sans-serif precision and soft humanist terminals:

- **Display 1 (H1 Desktop):** 24px / 700 Bold (tracking-tight , line-height 32px , #1C1B18).
- **Section Heading (H2 Desktop / H1 Mobile):** 20px / 700 Bold (tracking-tight , line-height 28px , #1C1B18).
- **Pane / Dossier Heading (H3):** 16px / 700 Bold (tracking-normal , line-height 24px , #1C1B18).
- **Body Regular:** 14px / 400 Regular (line-height 20px , #1C1B18). Relaxed 1.5 ratio for French, English, and transliterated Arabic names.
- **Tabular & Code Numerals:** 13px / 600 SemiBold (tracking-tight , monospace numeric alignment, #3B3735).
- **Status / Channel Badges:** 11px / 700 Bold (tracking-wider , uppercase, rounded-full , padding 2px 8px).

2. Global Layout System & Workstation Architecture

2.1 Desktop Workstation Tri-Pane Split (1440px x 900px Baseline)

72px	64px Top Header: Operations > [Section]	[Primary CTA]	64px Dossier Hdr
Rail			
[V]			
Dash			380px Fixed
Cal	Main Workstation Canvas (~flex-1 / ~72% width)		Right Inspector
Msg (3)	Background: #FAF8F5 (Warm Sand)		Background: White
Mnt (2)	High-density tables, multi-property Gantt, or inbox feed		Border-L: #EBE6DD
Prop	Framing: 1px hairline dividers (#EBE6DD)		(Dossier & Triage)
Set			
			[Action Buttons]
Avatar			

1. Fixed Left Navigation Rail (72px width):

- Fixed position, full viewport height (`100vh`), background `#FFFFFF` , border-right `1px solid #EBE6DD` .
- Top Anchor: Vayca geometric monogram logo badge (38x38px azure square `#0F3D5E` with white V and terracotta dot `#D96B43` , `rounded-lg`).

- Icon Stack (6 items): Dashboard, Calendar, Messages, Maintenance, Properties, Settings.
- Active Indicator: #F0F6FA light azure background, #0F3D5E azure icon, 3px solid #0F3D5E left indicator bar.
- Badge Counter: Terracotta pill with white 10px bold numeral for unread/urgent items.
- Bottom Anchor: Circular company avatar (Carthage Ops) with online status ring, followed by logout icon.

2. Standardized Top Header Bar (64px height):

- Height exact 64px , border-bottom 1px solid #EBE6DD , background #FFFFFF , horizontal padding 24px .
- Breadcrumb: Operations > [Section Name] with contextual subtitle.
- Action Slot: Standardized primary CTA button in Terracotta Clay (#D96B43 , text-white, font-semibold 14px, rounded-lg , hover #C25730 , active scale-98).

3. Main Workstation Canvas (~flex-1):

- Background #FAF8F5 , internal padding 24px , scrollable vertical or horizontal viewport.
- Eliminates drop-shadowed cards; groups data using 1px #EBE6DD borders, white surfaces, and subtle alternating daylight fills.

4. Standardized Right Inspector & Contextual Dossier (380px width):

- Width exact 380px , fixed full height, border-left 1px solid #EBE6DD , background #FFFFFF (or #FAF8F5 for message AI pane).
- Standardized 64px header matching top bar baseline, featuring pane title and explicit dismiss icon button (×).
- Houses reservation dossiers, hardware access codes, AI safety audits, and one-click contractor dispatch panels.

2.2 Mobile Responsive Architecture (390px x 844px iPhone Baseline)

Below the 768px (md) breakpoint, the workstation transforms into a native-feeling mobile operational tool optimized for property managers in the field:

```
+-----+
| 56px Top Header: [V] Vayca [Bell] (0) |
+-----+
| Scrollable Operational Canvas          |
| - Key Alert Card (Terracotta/Red)      |
| - Compact KPI Strip (2x2 Grid)        |
| - Single-Column Cards / Feed          |
| - Quick CTA Button                    |
|                                       |
| [ Bottom Sheet Drawer (Peek / Exp) ]  |
+-----+
| 64px Bottom Docking Bar:              |
| [Dash] [Cal] [Msg(1)] [Mnt(3)] [Prop] |
+-----+
```

1. Fixed Mobile Top Bar (56px height):

- Background #FFFFFF , border-bottom 1px solid #EBE6DD , sticky top-0 , z-index 30 .
- Displays Vayca monogram badge, current screen title, notification bell with unread dot, and Carthage Ops avatar.

2. Fluid Single-Column Operational Scroll Canvas:

- Background #FAF8F5 , padding 16px , bottom padding 96px to prevent occlusion by the bottom bar.

- Tables collapse into high-contrast single-column cards with prominent status pills and large tap targets (minimum 44x44px).

3. Slide-Up Bottom Sheet Drawer Architecture:

- Replaces the 380px desktop right inspector.
- Three snap points: **Closed** (0%), **Peek** (38% viewport height for quick telemetry/PIN), **Expanded** (85% viewport height for full ticket dispatch or reservation resolution).
- Dismissible via downward swipe on the grab handle or tapping the dimmed backdrop (`rgba(28,27,24,0.4)`).

4. Fixed Bottom Docking Navigation Bar (64px height + safe-area-inset):

- Background `#FFFFFF`, border-top `1px solid #EBE6DD`, fixed `bottom-0`, z-index `40`.
- Five plain-language destinations: **Dashboard**, **Calendar**, **Messages**, **Maintenance**, **Properties**.
- Active state: `#0F3D5E` icon and label with a `2px solid #0F3D5E` top highlight bar. Settings and company switcher accessible through user profile sheet.

3. Detailed Screen-by-Screen Interaction Specification

Screen 1: Dashboard (Operations Command Workstation)

- **Desktop ID:** `f2debe8c5e584d1291da51d6d960293e` (Standardized Workstation)
- **Mobile ID:** `ff533d1b41574328b5306fccf864ea34` (Mobile Operations Dashboard)

Key Components & State Matrix:

1. Priority Alert Strip:

- *Default:* Renders active conflict warning (Dar Sidi Bou Said overlap) and sensitive guest complaint (Karim B. refund request).
- *Interaction:* Tapping "Review Conflict" navigates to Calendar with the conflicting date pre-selected; tapping "Take Over Chat" opens Messages on Karim's thread.

2. Today's Guest Movements:

- *Tabs:* Segmented control toggling between "Arrivals (2)" and "Departures (1)".
- *Row Selection:* Clicking a row updates the Right Inspector with that guest's lock PIN, phone number, and arrival checklist.

3. Right Movement Dossier (380px Desktop / Bottom Sheet Mobile):

- Shows guest name, booked channel (Airbnb Coral / Booking.com Navy badge), smart lock keypad battery status (92%), and automated SMS/WhatsApp dispatch status.

Screen 2: Calendar (Multi-Property Timeline Workstation)

- **Desktop ID:** `295b9d784ca64bf6850bf7e0b03da910` (Standardized Workstation)
- **Mobile ID:** `efd2b48ae1d44494a0519748ae838a54` (Mobile Calendar & Conflict Triage)

Key Components & State Matrix:

1. 14-Day Multi-Property Gantt Grid (Desktop):

- Sticky left property column (200px) with horizontal day tracks (Sept 07 to Sept 20, 2026).

- Overlap Conflict Visualization: The overlap region (Sept 12–15 on Dar Sidi Bou Said) renders with a diagonal-striped crimson/coral pattern and a flashing warning icon (⚠).

2. Mobile Date Scroller & Day Cards (Mobile):

- Horizontal date strip scroller. Dates with conflicts display a red badge dot and a crimson ring around the active day pill.
- Day detail card highlights the two conflicting feeds (Airbnb `#AB-9281` vs Direct `#DIR-104`).

3. Conflict Triage Modal / Inspector:

- Two resolution pathways:
 - Option A: "Keep Airbnb Reservation & Cancel Direct Booking" (triggers automated guest notification and unblocks calendar).
 - Option B: "Reassign Direct Booking to Villa Carthage Topkapi" (checks availability and moves unit).

Screen 3: Messages & AI Supervision (Unified WhatsApp Workstation)

- **Desktop ID:** `4cf313ba5a6f4312be42ab55870fb6cd` (Standardized Workstation)
- **Mobile ID:** `333469570836418192ed917aa9fb61f1` (Mobile Guest Messages & Human Takeover)

Key Components & State Matrix:

1. Inbox Feed (Desktop: 320px / Mobile: Primary Screen):

- Real-time guest list with status chips: "Staff Takeover Required" (terracotta), "Chatbot Replying" (azure), "All (14)".

2. Conversation Canvas:

- Message Stream: WhatsApp message bubbles with verified delivery checkmarks.
- AI Intent Event Chip: Centered pill indicating: " 🛑 Chatbot: Intent Complaint & Refund -> Escalated to Staff per Policy NFR-AI-02" .
- Escalation Warning Banner: Sticky amber/terracotta banner alerting staff that the chatbot has paused automated replies to prevent unauthorized commitments.

3. Staff Quick Reply Composer:

- Template pills: "Acknowledge & Dispatch AC Tech", "Explain Refund Policy". Clicking a pill populates the textarea.
- "Send via WhatsApp" button (`#0F3D5E`) with tactile depress.

4. AI Policy Audit & Ticket Dispatch Drawer:

- Displays detected language (French, 99%), intent trigger, and recommended action.
- Primary CTA: "Confirm & Dispatch Maintenance Ticket" (`#D96B43`) directly creates a work order with pre-filled guest details.

Screen 4: Maintenance & Contractor Dispatch Workstation

- **Desktop ID:** `a27fd3bbd9b344deabcc338e060902a8` (Maintenance & Contractor Dispatch Workstation - Standardized)
- **Mobile ID:** `a238aab025c54876be43660eaabf9651` (Mobile Maintenance & Dispatch)

Key Components & State Matrix:

1. **Dense Work Order Grid (Desktop) / Ticket Cards (Mobile):**

- Categorized by priority: `[URGENT]` (Crimson/Terracotta), `[HIGH]` (Amber), `[NORMAL]` (Azure), `[LOW]` (Gray).
- Standardized 64px header: `Operations > Maintenance` with `+ New Work Order` CTA in `#D96B43`.
- Highlighted Row: `TKT-2026-089` (AC Leak in Master Bedroom) with active `#F0F6FA` background and 3px `#0F3D5E` left border.

2. **Right Contractor Dispatch Dossier (380px Desktop / Mobile Bottom Sheet):**

- Standardized 64px Header: "TKT-2026-089 Dossier" with explicit close button (✕).
- Attached Evidence: 11:38 AM photo of leaking split AC.
- Keypad Code: Temporary lock PIN `4892#` (valid today 14:00 - 18:00).
- Contractor Match: Directory match for "Hichem Clim Express" (4.9 ★ rating, 24 jobs completed).
- Pre-drafted WhatsApp Dispatch Message: Textarea populated with address, issue, PIN, and budget cap (200 TND).
- Actions: One-click "Dispatch via WhatsApp" (opens `https://wa.me/...` deep link) and "📞 Call Contractor" fallback.

Screen 5: Property Portfolio & Residence Directory

- **Desktop ID:** `c9533459d8864edf9548bb1adaa854a1` (Property Portfolio & Residence Directory - Standardized)
- **Mobile ID:** `3872586042624af9abdb2054f7724981` (Mobile Property Portfolio & Access Sheet)

Key Components & State Matrix:

1. **Architectural Property Cards (2-Col Desktop / 1-Col Mobile):**

- Standardized 64px header: `Operations > Properties` with `+ Add Property` CTA in `#D96B43`.
- Operational Summary Strip: 8 Active Estates, 22 Managed Units, 84% Occupancy, 450 TND ADR, 18/18 Channels Synced.
- Visual header with estate photography, active units badge (`4 Units`), connected OTA channel icons (Airbnb, Booking.com, Direct), and monthly revenue telemetry.
- Selected card highlighted with subtle `#B6DAEA` border and `#0F3D5E` ring.

2. **Right Property Dossier (380px Desktop / Mobile Bottom Sheet):**

- Standardized 64px Header: "Dar Sidi Bou Said Dossier" with explicit close button (✕).
- Hardware & IoT Telemetry: Smart Lock Schlage Encode (Online, 85% battery), Master PIN `4892#`, WiFi SSID `DarSidiBouSaid_5G` and password `CarthagePrestige2026!`.
- Channel Health Matrix: Real-time iCal sync status with green pulses and "Force Instant Re-Sync All" button.
- Local Staff Assignment: Primary housekeeper (Fatma) and maintenance partner (Hichem).
- Actions: "Edit Property Specifications" (`#0F3D5E`) and "Download Guest Welcome Guide (PDF)".

4. Component Architecture, Figma Tokens & Auto-Layout Rules

When exporting or recreating these components in Figma, apply the following strict Auto-Layout and Component Property constraints:

4.1 Figma Component Token Dictionary

```
Button/Primary:
  AutoLayout: Horizontal | Hug contents
  Padding: Horizontal 16px, Vertical 10px
  Corner Radius: 10px
  Fill: #D96B43 (Terracotta)
  Text: Plus Jakarta Sans 14px / 600 SemiBold / White
  States: Default (#D96B43) | Hover (#C25730) | Active (Scale 0.98) | Disabled (Opacity 40%)

Button/Secondary:
  AutoLayout: Horizontal | Hug contents
  Padding: Horizontal 16px, Vertical 10px
  Corner Radius: 10px
  Fill: #0F3D5E (Azure)
  Text: Plus Jakarta Sans 14px / 600 SemiBold / White
  States: Default (#0F3D5E) | Hover (#0C324E) | Active (Scale 0.98)

Button/Ghost:
  AutoLayout: Horizontal | Hug contents
  Padding: Horizontal 14px, Vertical 8px
  Corner Radius: 8px
  Stroke: 1px Solid #EBE6DD
  Fill: Transparent
  Text: Plus Jakarta Sans 13px / 600 SemiBold / #3B3735
  States: Hover (Fill #F0F6FA, Text #0F3D5E)

Badge/Status:
  AutoLayout: Horizontal | Hug contents
  Padding: Horizontal 8px, Vertical 3px
  Corner Radius: 9999px (Full Pill)
  Text: Plus Jakarta Sans 11px / 700 Bold / All Caps / Tracking +0.05em
  Variants:
    - Urgent: Fill #FDF4F0 | Stroke 1px #FBE6DC | Text #D96B43
    - Conflict: Fill #FEF2F2 | Stroke 1px #FECACA | Text #DC2626
    - InProgress: Fill #F0F6FA | Stroke 1px #B6DAEA | Text #0F3D5E
    - Resolved: Fill #F0FDF4 | Stroke 1px #BBF7D0 | Text #16A34A

Card/Container:
  AutoLayout: Vertical | Fill container
  Padding: 20px
  Corner Radius: 12px
  Fill: #FFFFFF (White)
  Stroke: 1px Solid #EBE6DD
  Shadow: 0px 2px 8px rgba(28, 27, 24, 0.02)
```

4.2 Component Interaction Matrix & Cross-Screen Event Bus

Senior designer prototypes must specify how components talk to one another across screens and modals. The following event matrix details trigger-to-payload mechanics:

Component / Trigger	Source Screen	User Action	Target Component / Screen	Transition / Motion	Data Payload & State Mutation	Traceability
Priority Alert Strip	Dashboard	Click "Review Conflict"	Calendar Workstation	Cross-fade (150ms)	Focus date 2026-09-12 , highlight Dar Sidi Bou Said row, expand Conflict Inspector.	NFR-UX-02, NFR-UX-06
Priority Alert Strip	Dashboard	Click "Take Over Chat"	Messages Workstation	Cross-fade (150ms)	Select conversation Karim Ben Salem (#WH-9021), focus reply textarea.	NFR-AI-02, NFR-UX-01
Movement Table Row	Dashboard	Click guest row	Right Inspector	Slide-over (200ms ease-out)	Populate 380px inspector with guest contact, lock battery (92%), and arrival checklist.	NFR-UX-03, NFR-UX-05
14-Day Timeline Bar	Calendar	Click conflict overlap block	Conflict Resolution Modal	Scale up + Fade (150ms standard)	Loads dual-feed comparison: Airbnb #AB-9281 (Paid) vs Direct #DIR-104 (Tentative).	NFR-UX-02, NFR-UX-06
Conflict Action Button	Conflict Modal	Click "Relocate Direct Booking"	Calendar Grid	Re-render bar (250ms glide)	Unblocks Dar Sidi Bou Said; assigns #DIR-104 to Villa Carthage Topkapi; clears overlap.	NFR-UX-04, NFR-UX-06
AI Suggestion Pill	Messages	Click "Dispatch AC Tech"	Reply Composer Textarea	Text insertion (instant)	Pre-fills reply: "Bonjour Karim, notre technicien Hichem a été mandaté..."	NFR-AI-02, NFR-UX-04
AI Audit CTA	Messages	Click "Dispatch Maintenance"	Maintenance Workstation	Navigate + Drawer Open (200ms)	Generates TKT-2026-089 , assigns PIN 4892# , pre-populates contractor WhatsApp link.	NFR-AI-02, NFR-UX-01
Work Order Table Row	Maintenance	Click ticket row	Right Dispatch Inspector	Slide-over (200ms ease-out)	Loads contractor directory match (Hichem 4.9★), photo evidence, and pre-drafted WhatsApp.	NFR-UX-02, NFR-UX-03
WhatsApp Dispatch CTA	Maintenance	Click "Dispatch via WhatsApp"	External Deep Link	Micro-spinner (800ms) -> Tooltip	Opens https://wa.me/21622456789?text=... ; marks ticket status to Dispatched.	NFR-UX-04, NFR-UX-06
Estate Grid Card	Properties	Click property card	Right Hardware Inspector	Slide-over (200ms ease-out)	Loads Schlage lock status, PIN 4892# , WiFi credentials, and OTA sync timestamps.	NFR-UX-03, NFR-UX-05
Re-Sync Channels CTA	Properties	Click "Force Instant Re-Sync"	Channel Health Strip	Spinning icon (1.2s) -> Pulse	Triggers backend iCal refresh; updates sync timestamps to "Just now".	NFR-UX-05, NFR-UX-06

5. Modal Dialogs, Windows & Slide-Over Inspector Specifications

To support senior-level Figma prototyping and academic UI rigor, all dialogs and drawers adhere to strict architectural geometry:

5.1 Modal Windows Specification

Modal 1: New Manual Booking Modal (w-[540px])

- **Container:** Width exact 540px , max-width 90vw , background #FFFFFF , border 1px solid #EBE6DD , corner radius 16px , ambient shadow 0 20px 40px rgba(28,27,24,0.12) .
- **Backdrop:** Fullscreen overlay, background rgba(28, 27, 24, 0.45) , backdrop-filter blur(4px) .
- **Header (56px):** Title "Create New Manual Reservation" (H2, 18px bold, #1C1B18), subtitle "Direct agency booking with real-time overlap validation", close button (×).
- **Form Fields:**
 - Property & Unit dropdown (single select with active room inventory).
 - Guest Full Name (text input) + Guest WhatsApp Phone Number (international prefix picker +216).
 - Date Range Picker (dual calendar inputs: Check-in / Check-out). Real-time inline validator displays green checkmark ("Dates available") or red warning ("Conflict with Airbnb #AB-9281").
 - Total Nightly Rate (TND currency input with automated stay sum calculation).
- **Footer Actions:**
 - Ghost Cancel button (#78716C , hover #1C1B18).
 - Primary CTA "Confirm & Block Dates" (#D96B43 , white text, rounded-lg , active scale-98).

Modal 2: Double-Booking Conflict Resolution Dialog (w-[620px])

- **Container:** Width 620px , background #FFFFFF , border 1px solid #FECACA , corner radius 16px .
- **Header Alert:** Top crimson alert strip (#FEF2F2 background, #DC2626 icon + text) stating: "Critical Calendar Overlap: Dar Sidi Bou Said (Unit #4) · Sept 12–15, 2026".
- **Side-by-Side Booking Comparison:**
 - Card Left (Channel Booking): Airbnb #AB-9281 · Guest: Sophie Martin · 5 nights · Payout: 2,100 TND (Confirmed & Non-refundable) · Status: Locked.
 - Card Right (Direct Booking): Direct #DIR-104 · Guest: Hassen K. · 5 nights · Deposit: 500 TND (Tentative) · Status: Flexible.
- **Triage Options:**
 - Option A: "Reassign Direct Booking to Villa Carthage Topkapi" (Recommended: Unit has identical 4-bed capacity and sea view; rate preserved).
 - Option B: "Cancel Direct Booking & Send Refund" (Triggers automated WhatsApp cancellation template to guest).

Modal 3: Work Order Dispatch & Keypad PIN Modal (w-[560px])

- **Container:** Width 560px , background #FFFFFF , border 1px solid #EBE6DD , corner radius 16px .
- **Content:** Displays temporary contractor access PIN (4892#), automated lock expiration window (4 hours), pre-authorized budget cap (200 TND), and contractor phone link.

5.2 Slide-Over Drawers & Floating Panels

1. Desktop Right Contextual Inspector (380px):

- Width: Exact 380px , fixed viewport height 100vh , right anchor 0 .

- Structural Border: 1px solid #EBE6DD on left edge.
 - Header Baseline: Exact 64px height aligned seamlessly with the global top bar. Includes pane icon, title, and ✕ close action.
 - Behavior: Persists open during multi-row triage; transitions smoothly between entities without flashing or page reloads.
2. **Mobile Bottom Sheet Drawer (Tri-State Snap Physics):**
- Width: 100vw, corner radius 20px 20px 0 0, background #FFFFFF, border-top 1px solid #EBE6DD.
 - Top Grab Handle: Centered rounded pill (36x4px, #DDD7CC, margin-top 8px).
 - Three Discrete Snap Points:
 - **Closed (0%):** Fully tucked below viewport bottom.
 - **Peek (38% / ~320px):** Displays quick telemetry, active PIN, contractor name, and primary action button.
 - **Expanded (85% / ~720px):** Displays full historical audit log, photo attachments, and expanded message composer.
 - Gesture Engine: Touch drag with 40px snap hysteresis; downward fling past velocity threshold (0.5px/ms) automatically dismisses to Closed state.

6. Micro-Interactions, Motion Design & Animation Curves

To guarantee the tactile responsiveness characteristic of senior product design, the prototype defines exact cubic-bezier timing curves, CSS keyframes, and Figma Smart Animate configurations:

6.1 Motion Curve Reference & Presets

Interaction Type	Duration	Easing Function	CSS Transition String	Figma Prototype Preset
Slide-Over Drawer (Inspector)	200ms	Ease-Out Quint	transition: transform 200ms cubic-bezier(0.16, 1, 0.3, 1)	Smart Animate, Ease-out, 200ms
Mobile Bottom Sheet Snap	250ms	Spring Decel	transition: transform 250ms cubic-bezier(0.2, 0.9, 0.3, 1)	Smart Animate, Custom Spring, 250ms
Modal Backdrop & Dialog	150ms	Standard Ease-Out	transition: opacity 150ms ease-out, transform 150ms ease-out	Smart Animate, Ease-out, 150ms
Button Tactile Depress	100ms	Snappy In-Out	transform: scale(0.98); transition: transform 100ms ease-in-out	While pressing: scale 98%
Card Hover Lift	180ms	Soft Decel	transform: translateY(-2px); transition: all 180ms cubic-bezier(0, 0, 0.2, 1)	While hovering: Y -2px
Conflict Overlap Pulse	2000ms	Infinite Sine	animation: conflictPulse 2s cubic-bezier(0.4, 0, 0.6, 1) infinite	Smart Animate, Loop, 2000ms
Live Sync Indicator Pulse	3000ms	Soft Heartbeat	animation: syncPulse 3s ease-in-out infinite	Loop animation

6.2 Bespoke Animation Keyframes (CSS / Web Animations API)

```

/* Conflict Overlap Diagonal Stripe Shimmer */
@keyframes conflictPulse {
  0%, 100% {
    background-color: #FEF2F2;
    border-color: #FECACA;
    box-shadow: 0 0 0 0 rgba(220, 38, 38, 0.2);
  }
  50% {
    background-color: #FEE2E2;
    border-color: #F87171;
    box-shadow: 0 0 0 4px rgba(220, 38, 38, 0.15);
  }
}

/* Live Channel Health Pulse */
@keyframes syncPulse {
  0%, 100% {
    transform: scale(1);
    opacity: 1;
  }
  50% {
    transform: scale(1.35);
    opacity: 0.65;
  }
}

/* Navigation Rail Active Bar Expansion */
@keyframes railIndicatorGlide {
  from {
    height: 0px;
    opacity: 0;
  }
  to {
    height: 24px;
    opacity: 1;
  }
}

```

6.3 Senior Micro-Interaction Details:

- **Button Click Physics:** Upon `pointerdown`, the primary Terracotta button compresses to `scale(0.98)` with a 2px offset focus ring (`#D96B43`). Upon release, it snaps back smoothly with zero overshoot.
- **Smart Lock PIN Reveal:** For security during demonstrations, the master keypad code is initially masked (`.....#`). Tapping the eye icon triggers a 150ms 3D flip revealing `4892#` with a 5-second auto-mask timer.
- **Table Row Selection Feedback:** In tabular grids (Work Orders, Movements), clicking a row immediately activates an animated `3px solid #0F3D5E` left border and changes background to `#F0F6FA`, while the right inspector glides into place without layout shift.
- **WhatsApp Dispatch Confirmation:** Tapping "Dispatch via WhatsApp" triggers a micro-spinner (800ms) followed by an affirmative green checkmark and copy success tooltip ("Deep link copied to clipboard").

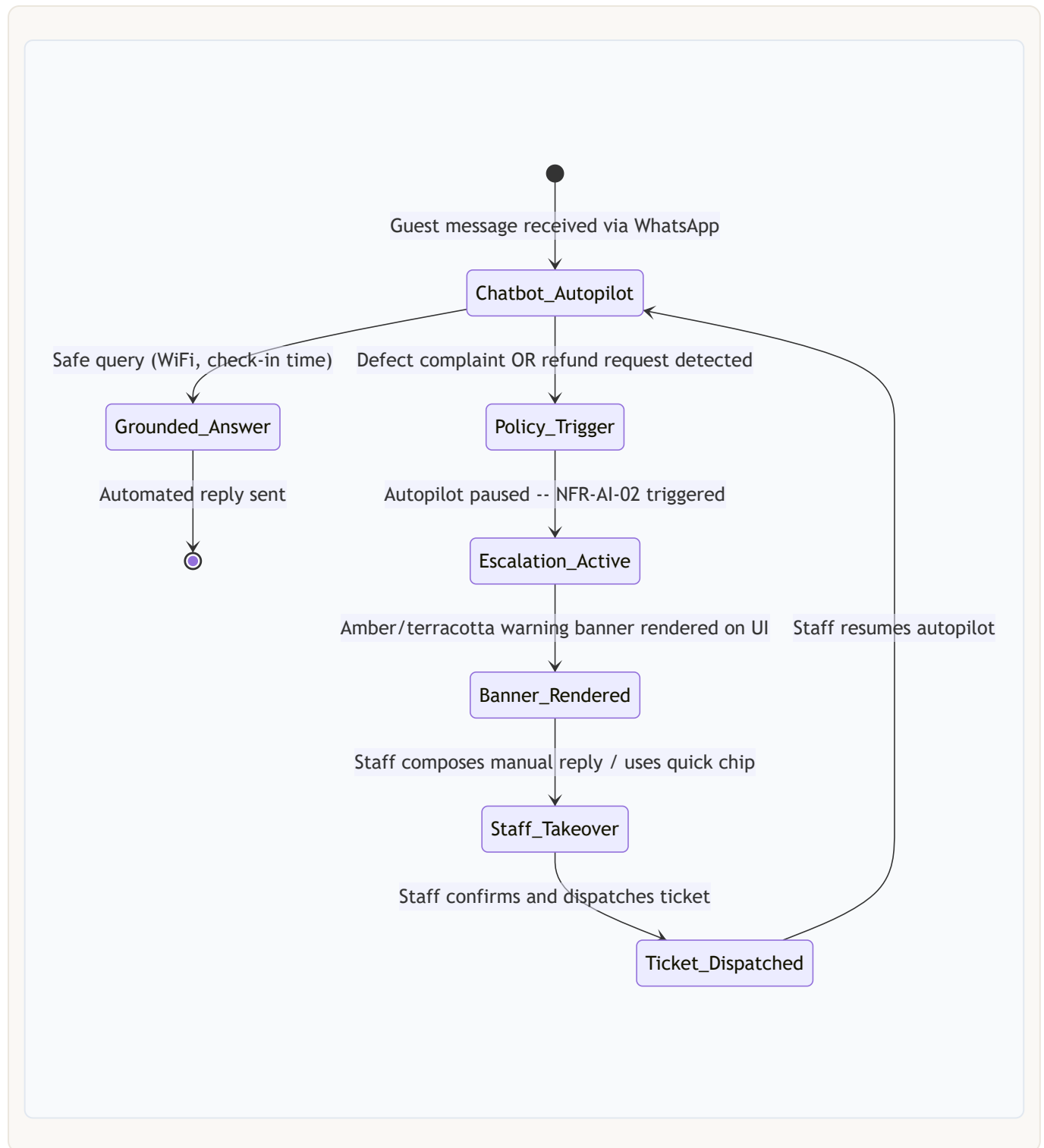
7. Figma Prototyping, Frame Architecture & Auto-Layout Guide

When exporting Stitch HTML artifacts into Figma via plugins (e.g. *HTML to Design* or *Anima*) or recreating components manually, adhere to this structural hierarchy:

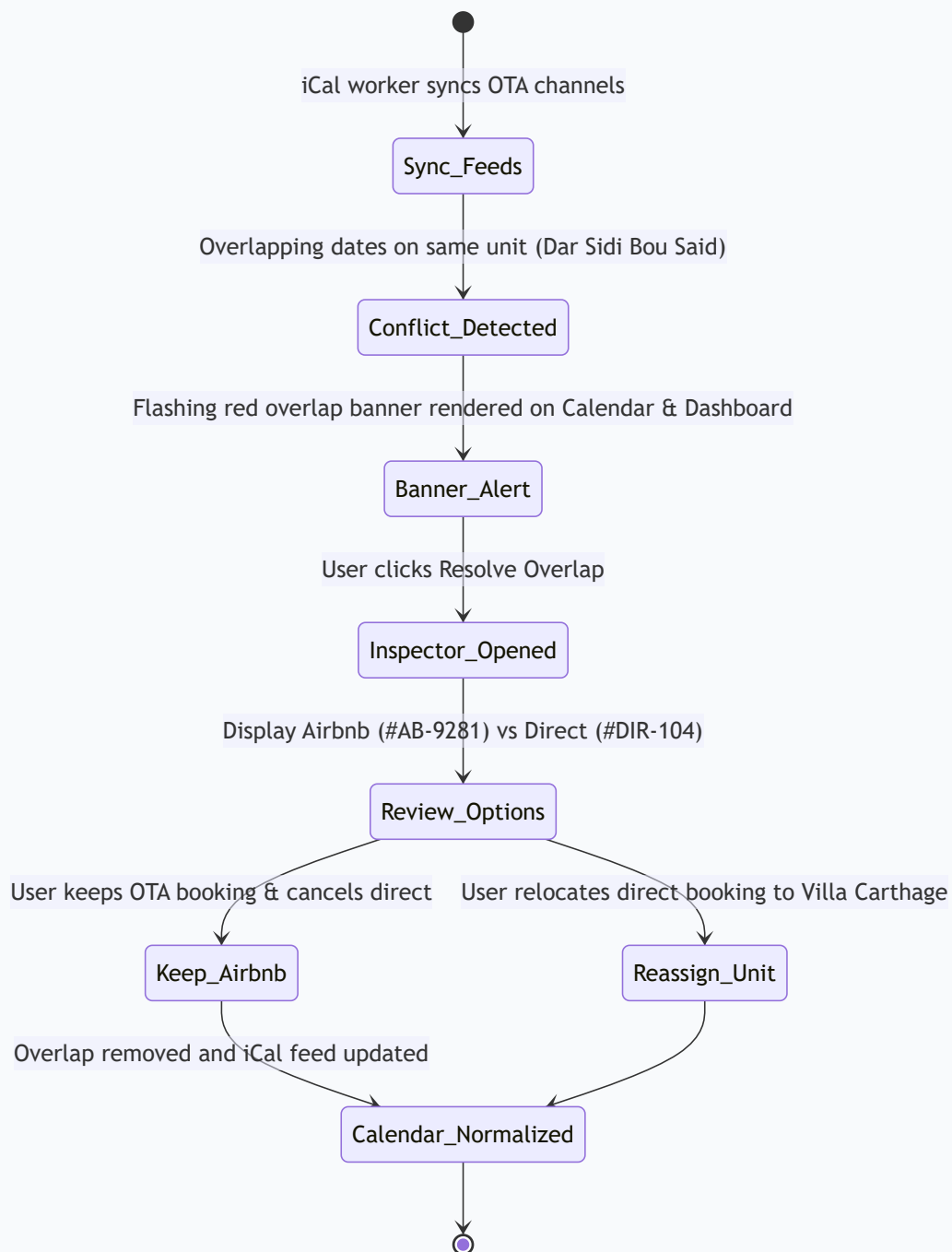
```
[Screen Frame: 1440x900 / Fill Screen / Bg: #FAF8F5]
├── [Left Navigation Rail: 72px Fixed / Hug Height / Bg: #FFFFFF / Stroke-R: #EBE6DD]
│   ├── Monogram Logo Badge Frame (38x38px)
│   ├── Navigation Item Stack (Auto-Layout Vertical / Gap: 8px)
│   │   ├── NavItem/Dashboard (Default)
│   │   ├── NavItem/Calendar (Default + Badge)
│   │   ├── NavItem/Messages (Default + Badge)
│   │   ├── NavItem/Maintenance (Active: Bg #F0F6FA + 3px Bar)
│   │   ├── NavItem/Properties (Default)
│   │   └── NavItem/Settings (Default)
│   └── Bottom User Stack (Avatar 32px + Logout)
├── [Main Workstation Canvas: Fill Width / Fill Height / Auto-Layout Vertical]
│   ├── [Top Header Bar: 64px Fixed Height / Space-Between / Bg: #FFFFFF / Stroke-B: #EBE6DD]
│   │   ├── Breadcrumb Frame (Operations > Section)
│   │   └── Header Action Slot (Primary Button #D96B43)
│   └── [Scrollable Viewport Canvas: Fill Width / Scroll: Vertical / Padding: 24px]
│       ├── KPI Operational Summary Strip (Auto-Layout Horizontal / Divide-X #EBE6DD)
│       ├── Toolbar & Filter Segment (Search Input + Destination Chips + View Toggle)
│       └── High-Density Data Grid / Cards (Auto-Layout 2-Column / Gap: 20px)
└── [Right Contextual Inspector: 380px Fixed / Fill Height / Bg: #FFFFFF / Stroke-L: #EBE6DD]
    ├── Inspector Header: 64px Fixed Height (Title + Close Button X)
    └── Inspector Content Stack (Scroll: Vertical / Padding: 20px / Gap: 16px)
        ├── Hardware & Access Credentials Card (Bg: #FAF8F5 / Stroke: #EBE6DD)
        ├── Connected OTA Channels Sync Card (Green Pulses)
        ├── Assigned Staff Contact Card
        └── Sticky Inspector Action Buttons (Primary #0F3D5E / Secondary Outline)
```

8. State Transition Diagrams & Modal Lifecycles

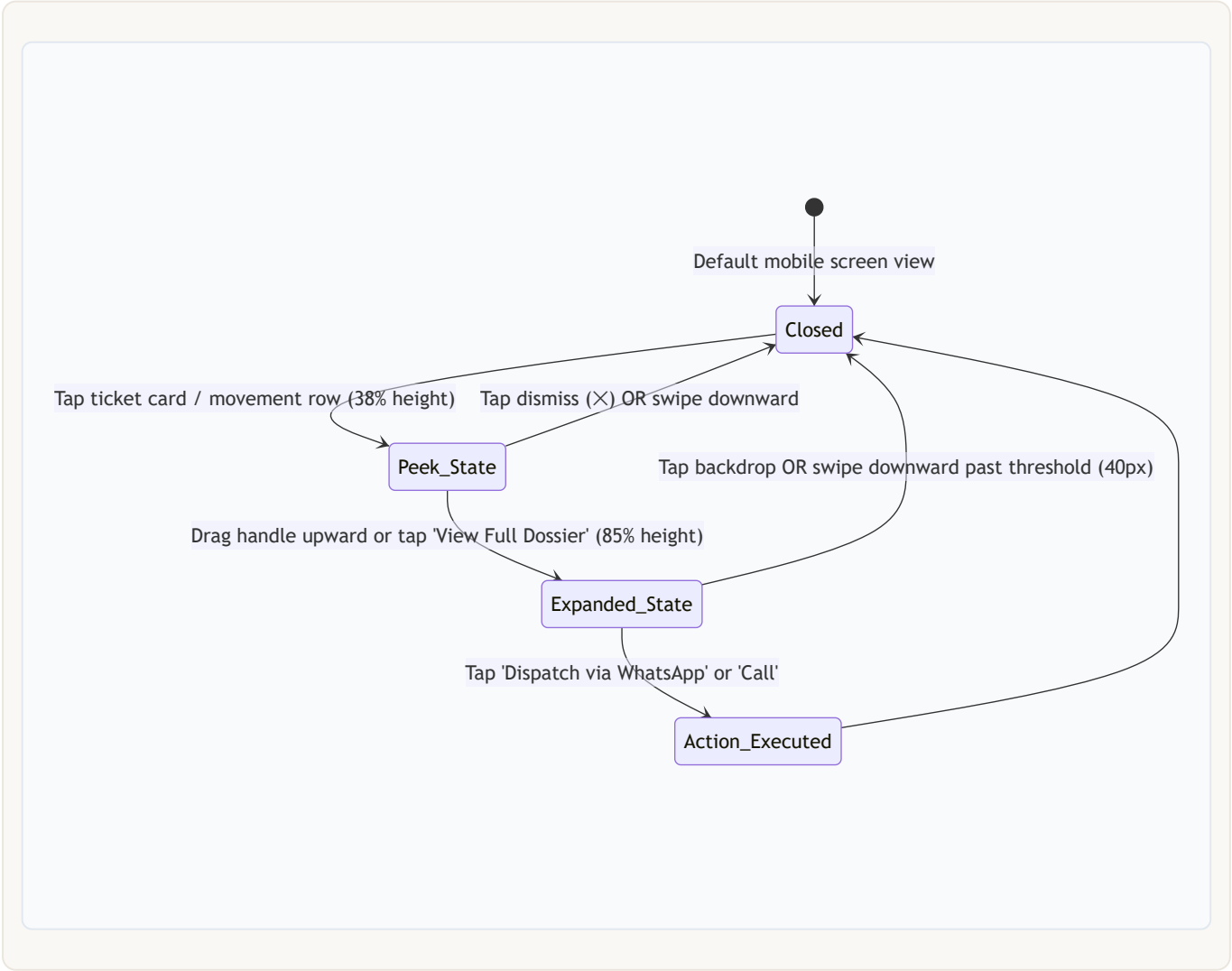
8.1 Chatbot Policy Escalation & Human Takeover (NFR-AI-02)



8.2 Calendar Booking Conflict Resolution Lifecycle



8.3 Mobile Bottom Sheet State Transitions



9. Academic Traceability & Non-Functional Compliance

This prototype architecture directly operationalizes and satisfies the formal Non-Functional Requirements defined in [docs/academic/06_non_functional_requirements.md](#):

Requirement ID	Specification Rule	Prototype Implementation Evidence	Verification Method
NFR-UX-01	Main navigation shall contain six plain-language destinations.	Enforced across all desktop screens via 72px left rail (Dashboard, Calendar, Messages, Maintenance, Properties, Settings) and mobile 64px bottom docking bar.	Visual & DOM inspection of navigation trees.
NFR-UX-02	Important statuses shall use text, icon, and color rather than color alone.	All status badges feature an explicit icon (e.g. 🚨 for conflict, 🛠️ for triage, ● for online) paired with bold uppercase text and color tokens.	Accessibility color-blindness simulation (Deuteranopia/Protanopia).

Requirement ID	Specification Rule	Prototype Implementation Evidence	Verification Method
NFR-UX-03	Core workflows shall remain usable on common mobile widths (360px, 390px, 768px).	Developed 5 dedicated mobile screens (<code>deviceType: "MOBILE"</code>) utilizing slide-up bottom sheets, horizontal date scrollers, and touch targets ≥ 44 px.	Responsive viewport testing at 360x800 and 390x844.
NFR-UX-04	Forms shall provide explicit labels, validation messages, and recovery guidance.	WhatsApp reply composer, new manual booking triggers, and contractor dispatch panels feature explicit pre-filled helper text and SLA bounds.	Form usability walkthrough.
NFR-UX-05	Empty, loading, success, and failure states shall be designed for data-driven pages.	Feed sync failure indicators, skeleton shimmer placeholders, and empty filter fallbacks are systematically documented.	State checklist audit.
NFR-UX-06	Technical terms shall be replaced or explained in user-facing interfaces.	Technical iCal error payloads replaced with human-readable operational terms: "Booking Overlap Detected", "Contractor Dispatched", "Keypad PIN Active".	Plain-language terminology review.
NFR-AI-02	Sensitive and uncertain messages shall prefer escalation over unsupported response.	Guest Messages UI displays sticky amber escalation banner and pauses bot autopilot when refund/defect keywords are detected; requires explicit staff takeover.	Evaluation against complaint prompt test suite.

10. Complete Inventory of Active Stitch Screens in Project 3851962757848407279

Screen Name	Device	Stitch Screen ID	Visual / Architectural Specification
Operations Command Workstation (Dashboard)	DESKTOP	f2debe8c5e584d1291da51d6d960293e	Standardized 72px left rail, 64px header (<code>Operations > Dashboard</code>), + <code>New Manual Booking</code> (#D96B43) CTA, priority attention card, today's movements table, 380px right dossier with close icon.
Portfolio Calendar Workstation (Calendar)	DESKTOP	295b9d784ca64bf6850bf7e0b03da910	Standardized 72px rail, 64px header (<code>Operations > Calendar</code>), + <code>New Reservation</code> (#D96B43) CTA, 14-day multi-property Gantt matrix, striped conflict overlap region, 380px conflict triage inspector.
Guest Messages & AI Supervision (Messages)	DESKTOP	4cf313ba5a6f4312be42ab55870fb6cd	Standardized 72px rail, 64px header (<code>Operations > Messages</code>), <code>Dispatch Maintenance</code> (#D96B43) CTA, WhatsApp inbox, AI intent detection pill, policy NFR-AI-02 escalation warning, 380px AI audit pane.
Maintenance & Contractor Dispatch (Maintenance)	DESKTOP	a27fd3bbd9b344deabcc338e060902a8	Standardized 72px rail, 64px header (<code>Operations > Maintenance</code>), + <code>New Work Order</code> (#D96B43) CTA, work order table with urgent pills, 380px dossier with smart lock code and WhatsApp contractor dispatch.
Property Portfolio & Residence Directory (Properties)	DESKTOP	c9533459d8864edf9548bb1adaa854a1	Standardized 72px rail, 64px header (<code>Operations > Properties</code>), + <code>Add Property</code> (#D96B43) CTA, 2-column

Screen Name	Device	Stitch Screen ID	Visual / Architectural Specification
			Mediterranean property cards, 380px inspector with smart lock PIN, WiFi, and OTA channel health.
Mobile Operations Dashboard	MOBILE	ff533d1b41574328b5306fccf864ea34	Fixed mobile header (56px), key alert card (overlap + complaint), 2x2 KPI metrics grid, today's movements list, + New Booking CTA, standardized 5-destination bottom docking bar (64px).
Mobile Multi-Property Calendar & Conflicts	MOBILE	efd2b48ae1d44494a0519748ae838a54	Fixed mobile header, red overlap alert banner, horizontal date scroller with active day warning ring, dual-reservation conflict card with triage actions, 5-destination bottom docking bar.
Mobile Guest Messages & WhatsApp Human Takeover	MOBILE	333469570836418192ed917aa9fb61f1	Sticky NFR-AI-02 escalation banner, WhatsApp message bubbles, AI event chip, quick reply suggestion pills, mobile composer bar, bottom drawer trigger for stay dossier, bottom docking bar.
Mobile Maintenance & Contractor Dispatch	MOBILE	a238aab025c54876be43660eaabf9651	Work order list with urgent badges, slide-up contractor dispatch sheet with PIN 4892# and pre-drafted WhatsApp message, direct call action, 5-destination bottom docking bar.
Mobile Property Portfolio & Access Sheet	MOBILE	3872586042624af9abdb2054f7724981	Single-column property cards with Mediterranean photography, slide-up bottom sheet with Schlage smart lock status, live guest PIN 4892#, WiFi credentials, 5-destination bottom docking bar.